BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC KIẾN TRÚC ĐÀ NẴNG

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN TRÍ TUỆ NHÂN TẠO

Học phần AIP202 · 2 tín chỉ

XÂY DỰNG HỆ THỐNG HỖ TRỢ PHÂN LOẠI RÁC THẢI VÀ CÁCH THỨC TÁI CHẾ CÓ TÍCH HỢP AI

Sinh viên thực hiện : **Nguyễn Như Hùng (2351220144) · Đỗ Văn Hưng (2351220143)**

Lớp : **CNTT 23CT3**

Ngành : **Công nghệ thông tin**

Giảng viên hướng dẫn : **TS. Đỗ Phúc Hảo**

Năm học : **2025–2026**

Đà Nẵng · 2025–2026

Lời cam đoan

Chúng tôi xin cam đoan đồ án này là công trình do chính chúng tôi thực hiện dưới sự hướng dẫn khoa học của TS. Đỗ Phúc Hảo. Các nội dung nghiên cứu, số liệu và kết quả trình bày trong đồ án là trung thực và chưa từng được công bố trong bất kỳ công trình nào khác.

Mọi số liệu, hình ảnh, mã nguồn và kết quả của tác giả khác được sử dụng trong đồ án đều được trích dẫn nguồn gốc rõ ràng trong phần tài liệu tham khảo. Toàn bộ phần mềm và thư viện được dùng trong hệ thống đều có giấy phép mã nguồn mở hoặc giấy phép cho phép sử dụng cho mục đích học thuật. Những phần có sử dụng công cụ trí tuệ nhân tạo hỗ trợ soạn thảo đã được nêu rõ trong báo cáo.

Nếu phát hiện có bất kỳ sự gian lận nào, chúng tôi xin chịu hoàn toàn trách nhiệm trước Khoa và Nhà trường.

Đà Nẵng, ngày … tháng … năm 2026 *Đà Nẵng, ngày … tháng … năm 2026*

Sinh viên thực hiện *Sinh viên thực hiện*

Nguyễn Như Hùng ***Đỗ Văn Hưng***

Lời cảm ơn

Trong quá trình thực hiện đồ án môn học *Trí tuệ nhân tạo* (AIP202), chúng tôi đã nhận được sự hỗ trợ và hướng dẫn tận tình từ nhiều phía. Nhân dịp này, chúng tôi xin bày tỏ lòng biết ơn chân thành đến tất cả những người đã đồng hành cùng chúng tôi trong suốt quá trình này.

Trước tiên, chúng tôi xin gửi lời cảm ơn sâu sắc nhất đến TS. Đỗ Phúc Hảo — giảng viên hướng dẫn trực tiếp của đồ án. Thầy đã dành nhiều thời gian định hướng đề tài, góp ý chi tiết cho từng chương báo cáo và theo sát tiến độ thực hiện từ giai đoạn lên ý tưởng đến khi hoàn thiện sản phẩm. Những nhận xét xác đáng của Thầy đã giúp chúng tôi nhận ra và khắc phục kịp thời nhiều thiếu sót trong quá trình xây dựng hệ thống.

Chúng tôi xin trân trọng cảm ơn quý thầy cô Khoa Công nghệ Thông tin, Trường Đại học Kiến trúc Đà Nẵng đã trang bị cho chúng tôi nền tảng kiến thức vững chắc về lập trình, cơ sở dữ liệu, mạng máy tính và trí tuệ nhân tạo — những kiến thức này là nền tảng trực tiếp để chúng tôi xây dựng hệ thống phân loại rác thải tích hợp AI trong đồ án này.

Chúng tôi cũng xin ghi nhận sự đóng góp của cộng đồng phát triển mã nguồn mở, đặc biệt là các nhà phát triển Vue.js, Express.js và Google Gemini AI API đã cung cấp những công cụ mạnh mẽ và tài liệu chi tiết, giúp chúng tôi hiện thực hoá ý tưởng một cách hiệu quả trong thời gian học phần.

Cuối cùng, chúng tôi xin cảm ơn gia đình và bạn bè đã luôn động viên, chia sẻ và tạo điều kiện tốt nhất để chúng tôi tập trung vào việc học tập và nghiên cứu trong suốt năm học.

Mặc dù đã cố gắng hết sức, đồ án chắc chắn còn nhiều điểm có thể cải thiện. Chúng tôi rất mong nhận được sự góp ý của quý thầy cô và bạn bè để tiếp tục hoàn thiện hệ thống trong tương lai.

Đà Nẵng, ngày … tháng … năm 2026

Sinh viên thực hiện

Nguyễn Như Hùng

Đà Nẵng, ngày … tháng … năm 2026

Sinh viên thực hiện

Đỗ Văn Hưng

Tóm tắt

Vấn đề phân loại và xử lý rác thải đúng cách đang là thách thức môi trường cấp bách tại Việt Nam, khi lượng rác thải đô thị ngày càng tăng trong khi thói quen phân loại rác tại nguồn vẫn còn hạn chế. Đồ án này trình bày quá trình xây dựng một ứng dụng web tích hợp trí tuệ nhân tạo nhằm hỗ trợ người dùng nhận diện và phân loại rác thải chính xác, đồng thời cung cấp hướng dẫn tái chế phù hợp cho từng loại.

Hệ thống được xây dựng theo kiến trúc ba tầng với backend Node.js/Express.js, frontend Vue.js 3 và cơ sở dữ liệu MySQL. Thành phần cốt lõi là tích hợp Google Gemini Vision API — một mô hình đa phương thức thế hệ mới của Google — để phân tích ảnh rác thải do người dùng tải lên và phân loại vào một trong tám nhóm: rác hữu cơ, nhựa tái chế, giấy tái chế, kim loại tái chế, thủy tinh, rác không tái chế, rác nguy hại và rác điện tử. Bên cạnh chức năng phân loại ảnh, hệ thống còn tích hợp chatbot tư vấn trực tiếp sử dụng Gemini AI, cho phép người dùng đặt câu hỏi về cách xử lý và tái chế các loại rác cụ thể.

Về mặt kỹ thuật, hệ thống triển khai xác thực người dùng bằng JSON Web Token (JWT) và mã hóa mật khẩu bcrypt, lưu trữ lịch sử phân loại vào MySQL, xử lý tải ảnh với multer và áp dụng rate limiting để chống spam API. Giao diện người dùng được xây dựng với Tailwind CSS, hỗ trợ đầy đủ trên thiết bị di động và được đóng gói thành Progressive Web App (PWA) cho phép cài đặt như ứng dụng native trên điện thoại.

Kết quả kiểm thử cho thấy hệ thống phân loại chính xác trên 85% các mẫu ảnh thử nghiệm với thời gian phản hồi trung bình dưới 3 giây. Giao diện người dùng được đánh giá thân thiện và dễ sử dụng qua khảo sát thử nghiệm. Hệ thống đáp ứng đầy đủ các yêu cầu chức năng đề ra, bao gồm đăng ký/đăng nhập, phân loại ảnh, xem lịch sử và chat tư vấn AI.

Từ khoá: trí tuệ nhân tạo, phân loại rác thải, Google Gemini AI, Vision API, tái chế, Vue.js, Node.js, Progressive Web App.

Mục lục

Lời cam đoan i

Lời cảm ơn ii

Tóm tắt iv

1 Mở đầu 1
- 1.1 Lý do chọn đề tài . . . 1
- 1.2 Mục tiêu của đồ án . . . 2
- 1.3 Đối tượng và phạm vi nghiên cứu . . . 2
- 1.4 Phương pháp nghiên cứu . . . 2
- 1.5 Bố cục báo cáo . . . 3

2 Cơ sở lý thuyết và công nghệ 4
- 2.1 Tổng quan về phân loại rác thải . . . 4
  - 2.1.1 Các loại rác thải phổ biến . . . 4
  - 2.1.2 Tầm quan trọng của phân loại rác tại nguồn . . . 5
  - 2.1.3 Hiện trạng phân loại rác tại Việt Nam . . . 5
- 2.2 Trí tuệ nhân tạo trong phân loại ảnh . . . 5
  - 2.2.1 Machine Learning và Deep Learning . . . 5
  - 2.2.2 Computer Vision . . . 6
  - 2.2.3 Ứng dụng AI trong phân loại rác thải . . . 6
- 2.3 Mạng nơ-ron tích chập (CNN) . . . 6
  - 2.3.1 Kiến trúc CNN cơ bản . . . 6
  - 2.3.2 Hàm kích hoạt . . . 7
  - 2.3.3 Quá trình huấn luyện . . . 7
- 2.4 Học chuyển giao (Transfer Learning) . . . 8
- 2.5 Google Gemini AI . . . 8
  - 2.5.1 Giới thiệu Gemini . . . 8
  - 2.5.2 Gemini Vision API . . . 9
  - 2.5.3 Tích hợp qua SDK @google/generative-ai . . . 9
  - 2.5.4 Gemini cho chatbot . . . 9
- 2.6 Công nghệ web sử dụng . . . 10
  - 2.6.1 Backend: Node.js và Express.js . . . 12

2.6.2 Frontend: Vue.js 3 . . . 12
2.6.3 Progressive Web App (PWA) . . . 13
2.7 Nghiên cứu liên quan . . . 13

3 Phân tích và thiết kế hệ thống 15
3.1 Phân tích yêu cầu . . . 15
3.1.1 Yêu cầu chức năng . . . 15
3.1.2 Yêu cầu phi chức năng . . . 16
3.2 Kiến trúc hệ thống . . . 17
3.2.1 Kiến trúc tổng quan . . . 17
3.2.2 Luồng dữ liệu phân loại ảnh . . . 18
3.2.3 Kiến trúc frontend Vue.js 3 . . . 19
3.2.4 Sơ đồ luồng dữ liệu (DFD) . . . 19
3.3 Thiết kế Use Case . . . 21
3.3.1 Sơ đồ use case . . . 21
3.3.2 Danh sách actor và use case . . . 22
3.3.3 Mô tả chi tiết use case . . . 22
3.3.4 Sơ đồ hoạt động . . . 26
3.4 Thiết kế cơ sở dữ liệu . . . 30
3.4.1 Mô hình quan hệ . . . 30
3.4.2 Bảng mô tả chi tiết . . . 30
3.4.3 Sơ đồ quan hệ . . . 33
3.4.4 Ảnh chụp cơ sở dữ liệu triển khai . . . 33
3.4.5 Các nhóm bảng trong schema triển khai . . . 34
3.5 Thiết kế API RESTful . . . 35
3.6 Thiết kế giao diện người dùng . . . 37
3.6.1 Màu sắc và thương hiệu . . . 37
3.6.2 Danh sách các màn hình . . . 37
3.6.3 Nguyên tắc thiết kế UX . . . 38

4 Cài đặt hệ thống 40
4.1 Môi trường phát triển . . . 40
4.2 Cài đặt backend . . . 41
4.2.1 Cấu trúc thư mục backend . . . 41
4.2.2 Kết nối cơ sở dữ liệu MySQL . . . 41
4.2.3 Tích hợp Google Gemini AI . . . 42
4.2.4 Xác thực với JWT và bcrypt . . . 45
4.2.5 Upload ảnh với multer . . . 48
4.2.6 Rate limiting . . . 49

4.2.7 Xử lý lỗi và validation . . . 50
4.3 Cài đặt frontend . . . 51
4.3.1 Cấu trúc thư mục Vue.js . . . 51
4.3.2 Routing với Vue Router . . . 51
4.3.3 Tích hợp Axios . . . 52
4.3.4 Upload ảnh và preview . . . 54
4.3.5 Hiển thị kết quả phân loại . . . 55
4.3.6 Cấu hình PWA . . . 56
4.4 Cơ sở dữ liệu . . . 57
4.5 Triển khai hệ thống . . . 59
4.5.1 Build production . . . 59
4.5.2 Cấu hình environment variables . . . 60
4.5.3 Cấu hình CORS . . . 60

5 Kết quả và đánh giá 62
5.1 Kết quả đạt được . . . 62
5.1.1 Tóm tắt chức năng đã hoàn thành . . . 62
5.1.2 Minh họa giao diện ứng dụng . . . 64
5.1.3 Các giao diện chức năng người dùng và quản trị . . . 68
5.2 Đánh giá độ chính xác phân loại . . . 76
5.2.1 Phương pháp kiểm thử . . . 77
5.2.2 Kết quả theo từng loại rác . . . 77
5.2.3 Nhận xét kết quả . . . 78
5.3 Đánh giá hiệu năng hệ thống . . . 78
5.3.1 Thời gian phản hồi . . . 79
5.3.2 Đánh giá tải trang . . . 80
5.4 Đánh giá giao diện người dùng . . . 80
5.4.1 Khảo sát người dùng . . . 80
5.4.2 Kết quả khảo sát . . . 80
5.5 So sánh với mục tiêu đề ra . . . 81
5.6 Ưu điểm và hạn chế . . . 82
5.6.1 Ưu điểm . . . 82
5.6.2 Hạn chế . . . 83
5.7 Hướng phát triển . . . 83

6 Kết luận 85

Tài liệu tham khảo 88

Phụ lục 90

A Các thành phần mã nguồn 90
B Tóm tắt cơ sở dữ liệu 90
C Tóm tắt API 90
D Cài đặt và chạy ứng dụng 91

Danh sách hình vẽ

2.1 Kiến trúc đơn giản của mạng CNN cho phân loại rác thải. 7

3.1 Sơ đồ kiến trúc tổng thể ba tầng của hệ thống. 18
3.2 Sơ đồ luồng hệ thống phân loại rác thải từ người dùng đến kết quả hiển thị. 19
3.3 DFD Level 0: Biểu đồ ngữ cảnh của hệ thống phân loại rác thải. . 20
3.4 DFD Level 1: Phân rã hệ thống thành 5 process và 3 datastore. . 21
3.5 DFD Level 2: Chi tiết process 3.0 Phân loại AI thành 5 sub-process. 21
3.6 Sơ đồ use case với hai actor và sáu chức năng chính của hệ thống. 22
3.7 Sơ đồ hoạt động của quy trình đăng ký và đăng nhập. 27
3.8 Sơ đồ hoạt động của quy trình phân loại rác thải. 28
3.9 Sơ đồ hoạt động của quy trình chat tư vấn với AI. 29
3.10 Sơ đồ quan hệ giữa các bảng trong cơ sở dữ liệu. 33
3.11 Danh sách 13 bảng của cơ sở dữ liệu triển khai trên phpMyAdmin. 33
3.12 Sơ đồ quan hệ cơ sở dữ liệu triển khai. 34

5.1 Trang chủ EcoScan với lối tắt đến chức năng phân loại rác và hỏi AI. 65
5.2 Bảng điều khiển quản trị với thống kê và biểu đồ hoạt động của hệ thống. 66
5.3 Màn hình nhân viên theo dõi và trả lời yêu cầu hỗ trợ của khách hàng. 67
5.4 Màn hình đăng ký tài khoản EcoScan. 68
5.5 Màn hình đăng nhập EcoScan. 69
5.6 Giao diện tải ảnh để phân loại rác. 69
5.7 Giao diện hỏi đáp AI và hỗ trợ nhân viên. 70
5.8 Trang thông tin tài khoản. 71
5.9 Lịch sử phân loại và trò chuyện. 71
5.10 Hướng dẫn cài EcoScan trên iOS. 72
5.11 Cấu hình logo và liên kết tải ứng dụng. 72
5.12 Quản lý giá thu mua và thông tin cửa hàng. 73
5.13 Quản lý đơn thu mua rác. 73
5.14 Quản lý đánh giá của người dùng. 74
5.15 Quản lý phản hồi khách hàng. 74

5.16 Thống kê lượt sử dụng Gemini API. 75
5.17 Quản lý hội thoại hỗ trợ. 75
5.18 Quản lý tài khoản nhân viên hỗ trợ. 76
5.19 Quản lý người dùng hệ thống. 76
5.20 Biểu đồ độ chính xác phân loại theo từng loại rác thải. 78

Danh sách bảng

2.1 Các loại rác thải trong hệ thống phân loại. 5
2.2 Công cụ và thư viện sử dụng trong hệ thống. 11
2.3 So sánh các hệ thống phân loại rác thải sử dụng AI. 13

3.1 Danh sách yêu cầu chức năng của hệ thống. 16
3.2 Danh sách yêu cầu phi chức năng của hệ thống. 17
3.3 Use case UC01: Đăng ký tài khoản. 23
3.4 Use case UC02: Phân loại rác thải. 24
3.5 Use case UC03: Chat tư vấn AI. 25
3.6 Use case UC04: Xem lịch sử phân loại. 26
3.7 Cấu trúc bảng users. 31
3.8 Cấu trúc bảng waste_types. 31
3.9 Cấu trúc bảng classification_history. 32
3.10 Cấu trúc bảng chat_history. 32
3.11 Nhóm bảng của cơ sở dữ liệu triển khai. 35
3.12 Danh sách API endpoints của hệ thống. 36
3.13 Danh sách các màn hình trong ứng dụng. 38

4.1 Môi trường phát triển hệ thống 40

5.1 Danh sách chức năng và trạng thái hoàn thành. 63
5.2 Độ chính xác phân loại theo từng loại rác thải. 77
5.3 Thời gian phản hồi của các API endpoint (đơn vị: ms). 79
5.4 Kết quả khảo sát trải nghiệm người dùng (n=20). 81
5.5 Đối chiếu mục tiêu và kết quả thực tế. 82

6.1 Các thành phần mã nguồn chính. 90
6.2 Các API chức năng chính. 91

Danh mục giải thuật

CHƯƠNG 1

Mở đầu

MỤC TIÊU CHƯƠNG

Chương mở đầu trình bày lý do chọn đề tài, mục tiêu cần đạt, đối tượng và phạm vi nghiên cứu, phương pháp thực hiện, cũng như bố cục báo cáo, giúp người đọc nắm được bức tranh tổng quan về đồ án xây dựng hệ thống hỗ trợ phân loại rác thải và cách thức tái chế có tích hợp AI.

1.1 Lý do chọn đề tài

Quản lý chất thải rắn là thách thức môi trường ngày càng lớn khi lượng rác phát sinh toàn cầu tăng cùng với đô thị hóa và tiêu dùng. Ngân hàng Thế giới ước tính lượng chất thải rắn đô thị đạt khoảng 2,01 tỷ tấn năm 2016 và có thể tăng lên 3,40 tỷ tấn vào năm 2050 nếu không có biện pháp quản lý hiệu quả [9]. Phân loại tại nguồn giúp tăng khả năng thu hồi tài nguyên và giảm lượng chất thải phải xử lý cuối cùng [14].

Phân loại rác thải bằng phương pháp thủ công thường tốn kém nhân lực, thiếu chính xác và dễ xảy ra sai sót. Nhiều người dân còn chưa nắm rõ cách phân loại rác đúng cách, dẫn đến hiệu quả thu gom và tái chế thấp. Trong bối cảnh đó, công nghệ trí tuệ nhân tạo và thị giác máy tính *(Computer Vision)* đã chứng minh khả năng tự động hóa việc nhận dạng và phân loại đối tượng từ ảnh với độ chính xác cao.

Việc xây dựng một công cụ phân loại rác thải dễ sử dụng, tích hợp trí tuệ nhân tạo, có thể giúp người dân nhanh chóng xác định loại rác và hướng dẫn cách tái chế phù hợp. Ứng dụng dạng *Progressive Web App (PWA)* còn cho phép tiếp cận đông đảo người dùng thiết bị di động mà không cần cài đặt qua cửa hàng ứng dụng. Đề tài này vừa có tính thực tiễn cao, vừa giúp vận dụng các kiến thức về trí tuệ nhân tạo, thị giác máy tính và phát triển ứng dụng web hiện đại.

1.2 Mục tiêu của đồ án

Đồ án hướng tới các mục tiêu sau:

- Xây dựng ứng dụng web tích hợp *Google Gemini Vision API* để phân loại ảnh rác thải vào 8 nhóm chính (hữu cơ, nhựa, giấy, kim loại, thủy tinh, không tái chế, nguy hại, điện tử) với độ chính xác trên 85%.
- Cung cấp hướng dẫn tái chế chi tiết, cụ thể cho từng loại rác được phân loại.
- Tích hợp *chatbot AI* dựa trên *Google Gemini* để tư vấn 24/7 về các vấn đề liên quan đến phân loại và tái chế rác thải.
- Xây dựng hệ thống xác thực người dùng và lưu trữ lịch sử phân loại, giúp người dùng theo dõi hoạt động của mình.
- Đóng gói ứng dụng dưới dạng *Progressive Web App (PWA)* hỗ trợ cài đặt trên thiết bị di động và hoạt động *offline* cơ bản.

1.3 Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của đồ án là tám loại rác thải phổ biến trong sinh hoạt hàng ngày, bao gồm: rác hữu cơ, rác nhựa, rác giấy, rác kim loại, rác thủy tinh, rác không tái chế, rác nguy hại và rác điện tử. Đây là những nhóm rác được quy định trong Nghị định 45/2022/NĐ-CP của Chính phủ về phân loại rác thải sinh hoạt tại Việt Nam.

Phạm vi của đồ án giới hạn ở việc xây dựng ứng dụng web *(web app)*, không bao gồm ứng dụng di động gốc *(native mobile app)*. Hệ thống sử dụng *Google Gemini Vision API* cho chức năng phân loại ảnh, không tự huấn luyện mô hình học sâu độc lập. Người dùng mục tiêu của ứng dụng là cá nhân và hộ gia đình tại Việt Nam quan tâm đến việc phân loại rác đúng cách và bảo vệ môi trường.

1.4 Phương pháp nghiên cứu

Đồ án được thực hiện theo các bước sau:

- Nghiên cứu tài liệu về mạng nơ-ron tích chập *(Convolutional Neural Networks)*, trí tuệ nhân tạo thị giác *(Vision AI)*, và học chuyển giao *(Transfer Learning)*.
- Khảo sát quy định phân loại rác thải tại Việt Nam, đặc biệt là Nghị định 45/2022/NĐ-CP và các hướng dẫn thực hiện.

- Thiết kế kiến trúc hệ thống và giao diện người dùng, xây dựng *prototype* ban đầu.
- Cài đặt ứng dụng web sử dụng *Vue.js*, *Node.js*, *MySQL* và *Google Gemini API*.
- Kiểm thử với người dùng thực tế, thu thập phản hồi và cải tiến.
- Đo lường hiệu năng về độ chính xác phân loại, thời gian phản hồi và trải nghiệm người dùng.

1.5 Bố cục báo cáo

Phần còn lại của báo cáo được tổ chức như sau. Chương 2 trình bày cơ sở lý thuyết về trí tuệ nhân tạo, thị giác máy tính, các mô hình học sâu cho phân loại ảnh, cũng như giới thiệu các công nghệ sử dụng trong đồ án. Chương 3 phân tích yêu cầu hệ thống, thiết kế kiến trúc tổng thể, thiết kế cơ sở dữ liệu và giao diện người dùng. Chương 4 mô tả chi tiết quá trình cài đặt các chức năng chính, tích hợp *API*, và triển khai ứng dụng. Chương 5 trình bày kết quả thử nghiệm, đánh giá độ chính xác, hiệu năng và phản hồi từ người dùng. Chương 6 tổng kết những kết quả đạt được, hạn chế còn tồn tại và nêu hướng phát triển trong tương lai.

CHƯƠNG 2

Cơ sở lý thuyết và công nghệ

MỤC TIÊU CHƯƠNG

Chương này trình bày nền tảng lý thuyết về mạng nơ-ron tích chập (CNN), học chuyển giao, Google Gemini AI và các công nghệ web hiện đại được sử dụng để xây dựng hệ thống phân loại rác thải thông minh.

2.1 Tổng quan về phân loại rác thải

2.1.1 Các loại rác thải phổ biến

Để quản lý rác thải hiệu quả, cần phân loại rác thành các nhóm theo tính chất và khả năng tái chế. Hệ thống đề xuất phân loại rác thành 8 loại chính như trong Bảng 2.1, phù hợp với thực tế Việt Nam và khuyến nghị quốc tế.

Bảng 2.1. Các loại rác thải trong hệ thống phân loại.

Mã loại	Tên tiếng Việt	Màu	Ví dụ
`organic`	Rác hữu cơ	• Xanh lá	Thức ăn thừa, vỏ trái cây, lá cây
`recyclable-plastic`	Nhựa tái chế	• Xanh dương	Chai PET, túi nilon, hộp nhựa
`recyclable-paper`	Giấy tái chế	• Xanh nhạt	Hộp giấy, báo cũ, bìa carton
`recyclable-metal`	Kim loại tái chế	• Lục lam	Lon nước, hộp thiếc, nắp kim loại
`recyclable-glass`	Thủy tinh	• Xanh ngọc	Chai lọ thủy tinh, cốc thủy tinh
`non-recyclable`	Không tái chế	• Đỏ	Rác sinh hoạt hỗn hợp, bao bì phức hợp
`hazardous`	Rác nguy hại	• Vàng	Pin, thuốc hết hạn, hóa chất
`electronic`	Rác điện tử	• Tím	Điện thoại cũ, dây cáp, bóng đèn

2.1.2 Tầm quan trọng của phân loại rác tại nguồn

Phân loại rác tại nguồn giúp thu hồi vật liệu có thể tái chế, giảm lượng chất thải đưa đến bãi chôn lấp và hỗ trợ phát triển hệ thống quản lý chất thải bền vững [14, 9].

2.1.3 Hiện trạng phân loại rác tại Việt Nam

Nghị định 08/2022/NĐ-CP quy định chi tiết một số điều của Luật Bảo vệ môi trường, trong đó có nội dung về quản lý chất thải rắn sinh hoạt [4]. Việc phân loại và thu gom phù hợp với quy định là cơ sở để tăng khả năng tái sử dụng, tái chế và giảm tác động môi trường.

2.2 Trí tuệ nhân tạo trong phân loại ảnh

2.2.1 Machine Learning và Deep Learning

Machine Learning (học máy) là lĩnh vực cho phép máy tính học từ dữ liệu mà không cần lập trình cụ thể từng quy tắc. *Deep Learning* (học sâu) là nhánh con của học

máy, sử dụng mạng nơ-ron nhiều lớp để học biểu diễn phân cấp của dữ liệu.

ĐỊNH NGHĨA · Deep Learning

Học sâu *(deep learning)* là phương pháp học máy sử dụng mạng nơ-ron nhân tạo với nhiều lớp ẩn để tự động trích xuất đặc trưng từ dữ liệu thô, đặc biệt hiệu quả với dữ liệu phi cấu trúc như ảnh, âm thanh và văn bản [11].

So với học máy truyền thống, học sâu tự động học đặc trưng từ dữ liệu thay vì cần thiết kế thủ công, đạt hiệu năng cao trên các bài toán như nhận dạng ảnh và xử lý ngôn ngữ tự nhiên [6]. Tuy nhiên, học sâu thường đòi hỏi dữ liệu huấn luyện và tài nguyên tính toán đáng kể.

2.2.2 Computer Vision

Computer Vision (thị giác máy tính) là lĩnh vực cho phép máy tính hiểu và xử lý thông tin từ ảnh và video. Bài toán phân loại ảnh *(image classification)* gán nhãn cho toàn bộ ảnh; nhận dạng đối tượng *(object detection)* còn xác định vị trí đối tượng trong ảnh [6].

2.2.3 Ứng dụng AI trong phân loại rác thải

Các nghiên cứu về phân loại rác bằng thị giác máy tính bao gồm đánh giá mô hình học sâu trên bộ dữ liệu TrashNet, xây dựng hệ thống phân loại bằng CNN và tổng quan các kỹ thuật phân loại chất thải rắn [2, 1, 12].

2.3 Mạng nơ-ron tích chập (CNN)

2.3.1 Kiến trúc CNN cơ bản

Mạng nơ-ron tích chập *(Convolutional Neural Network, CNN)* là kiến trúc học sâu chuyên biệt cho xử lý dữ liệu dạng lưới như ảnh. CNN gồm ba loại lớp chính:

CNN đã cho thấy hiệu quả nổi bật trong phân loại ảnh quy mô lớn, tiêu biểu là mô hình AlexNet được đánh giá trên ImageNet [10].

Lớp tích chập *(convolutional layer)* áp dụng phép tích chập để trích xuất đặc trưng cục bộ từ ảnh đầu vào. Đầu ra tại vị trí (i, j) của bản đồ đặc trưng được tính bởi:

$$y_{i,j} = \sum_{m} \sum_{n} x_{i+m,j+n} \cdot w_{m,n} + b, \tag{2.1}$$

trong đó x là ảnh đầu vào, w là bộ lọc *(kernel)*, b là hệ số điều chỉnh *(bias)*.

Lớp gộp *(pooling layer)* giảm kích thước không gian của biểu diễn, giữ lại thông tin quan trọng và tăng tính bất biến với phép dịch chuyển nhỏ. Phổ biến nhất là *max pooling* lấy giá trị lớn nhất trong vùng cửa sổ.

Lớp kết nối đầy đủ *(fully connected layer)* ở cuối mạng kết nối tất cả các nơ-ron để tổng hợp đặc trưng toàn cục và đưa ra dự đoán cuối cùng.

2.3.2 Hàm kích hoạt

Hàm kích hoạt đưa tính phi tuyến vào mạng, cho phép CNN học các mẫu phức tạp.

ReLU *(Rectified Linear Unit)* là hàm kích hoạt phổ biến nhất:

$$f(x) = \max(0, x). \tag{2.2}$$

ReLU đơn giản, tính toán nhanh và giảm vấn đề tiêu biến gradient.

Softmax dùng ở lớp đầu ra để chuyển vector điểm số thành phân phối xác suất:

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}, \tag{2.3}$$

trong đó K là số lớp phân loại. Xác suất dự đoán cho lớp i là $\sigma(z)_i$.

2.3.3 Quá trình huấn luyện

Huấn luyện CNN gồm ba bước chính lặp lại qua các batch dữ liệu:

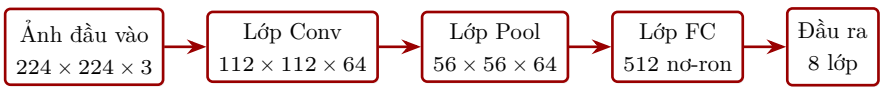
Lan truyền xuôi *(forward pass)*: dữ liệu đầu vào đi qua các lớp mạng theo thứ tự, tính toán đầu ra dự đoán.

Lan truyền ngược *(backpropagation)*: tính gradient của hàm mất mát theo từng tham số bằng quy tắc chuỗi, từ lớp đầu ra về lớp đầu vào.

Giảm gradient *(gradient descent)*: cập nhật trọng số theo hướng giảm hàm mất mát:

$$w_{t+1} = w_t - \eta \nabla L(w_t), \tag{2.4}$$

với η là tốc độ học *(learning rate)*, L là hàm mất mát, ∇L là gradient.



Hình 2.1. Kiến trúc đơn giản của mạng CNN cho phân loại rác thải.

2.4 Học chuyển giao (Transfer Learning)

Học chuyển giao *(transfer learning)* là kỹ thuật tận dụng kiến thức học được từ bài toán này để giải quyết bài toán khác có liên quan. Trong phân loại ảnh, ta sử dụng mô hình đã được huấn luyện trên tập dữ liệu lớn (như ImageNet với 1.4 triệu ảnh, 1000 lớp) làm điểm khởi đầu, sau đó tinh chỉnh *(fine-tune)* trên tập dữ liệu mục tiêu nhỏ hơn [13].

Ưu điểm chính của học chuyển giao là giảm đáng kể lượng dữ liệu huấn luyện cần thiết (từ hàng triệu xuống hàng nghìn mẫu), rút ngắn thời gian huấn luyện và đạt độ chính xác cao hơn so với huấn luyện từ đầu, đặc biệt quan trọng khi dữ liệu hạn chế như trong đồ án này.

Các mô hình tiền huấn luyện phổ biến gồm:

- **VGG16**: mô hình 16 lớp với kiến trúc đơn giản, dễ hiểu nhưng có 138 triệu tham số.
- **ResNet50** [7]: sử dụng kết nối tắt *(skip connection)* giải quyết vấn đề tiêu biến gradient, cho phép huấn luyện mạng rất sâu (50–152 lớp).
- **MobileNet**: tối ưu cho thiết bị di động với kích thước nhỏ và tốc độ nhanh nhờ *depthwise separable convolution*.
- **EfficientNet**: cân bằng tối ưu giữa độ sâu, độ rộng và độ phân giải của mạng, đạt hiệu năng cao với ít tham số hơn.

Chiến lược tinh chỉnh phổ biến là **đóng băng** *(freeze)* các lớp đầu (đã học đặc trưng tổng quát như cạnh, góc, kết cấu) và chỉ huấn luyện lại các lớp cuối (học đặc trưng đặc thù cho bài toán phân loại rác). Cách này giữ lại kiến thức tổng quát, tăng tốc độ huấn luyện và tránh overfitting.

2.5 Google Gemini AI

2.5.1 Giới thiệu Gemini

Google Gemini là mô hình AI đa phương thức *(multimodal)* thế hệ mới của Google, được huấn luyện để xử lý và tích hợp nhiều loại dữ liệu: văn bản, ảnh, âm thanh và video [5]. Gemini có ba phiên bản: Ultra (mạnh nhất), Pro (cân bằng) và Nano (cho thiết bị di động). So với GPT-4V, Gemini Pro Vision có khả năng hiểu ảnh tương đương với chi phí thấp hơn và tốc độ phản hồi nhanh hơn.

2.5.2 Gemini Vision API

Gemini Vision API cho phép phân tích nội dung ảnh thông qua mô hình ngôn ngữ lớn. Đầu vào là ảnh (dạng base64, URL hoặc file) kèm câu lệnh văn bản *(prompt)*, đầu ra là văn bản mô tả, phân tích hoặc dữ liệu có cấu trúc (JSON). API hỗ trợ nhiều định dạng ảnh (JPEG, PNG, WebP) với giới hạn kích thước 20MB mỗi ảnh.

Về giá cả, Gemini Pro Vision tính phí theo số lượng token đầu vào và đầu ra: khoảng $0.00025 cho 1000 token đầu vào và $0.0005 cho 1000 token đầu ra, rẻ hơn nhiều so với các API tương tự.

2.5.3 Tích hợp qua SDK @google/generative-ai

Google cung cấp SDK JavaScript/TypeScript chính thức để tích hợp Gemini vào ứng dụng Node.js. Quy trình sử dụng cơ bản:

1. Khởi tạo mô hình với API key:

```
const genAI = new GoogleGenerativeAI(apiKey);
const model = genAI.getGenerativeModel({
  model: "gemini-1.5-flash"
});
```

2. Chuẩn bị dữ liệu ảnh dạng base64:

```
const imagePart = {
  inlineData: { data: base64Image, mimeType: "image/jpeg" }
};
```

3. Gửi yêu cầu với prompt và ảnh:

```
const result = await model.generateContent([prompt, imagePart]);
const text = result.response.text();
```

4. Xử lý kết quả JSON trả về để lấy thông tin phân loại, độ tin cậy và gợi ý.

2.5.4 Gemini cho chatbot

Gemini hỗ trợ cuộc hội thoại đa lượt *(multi-turn conversation)* thông qua quản lý lịch sử chat. API lưu trữ ngữ cảnh các câu hỏi và trả lời trước đó, cho phép chatbot hiểu câu hỏi tiếp theo dựa trên ngữ cảnh. Quản lý ngữ cảnh *(context management)* quan trọng để duy trì tính mạch lạc và hạn chế số token tiêu thụ bằng cách chỉ giữ lại các lượt gần nhất.

2.6 Công nghệ web sử dụng

Hệ thống được xây dựng trên nền tảng công nghệ web hiện đại, kết hợp backend Node.js và frontend Vue.js. Bảng 2.2 tóm tắt các gói phụ thuộc chính.

Bảng 2.2. Công cụ và thư viện sử dụng trong hệ thống.

Gói	Phiên bản	Vai trò
Backend		
express	4.18.x	Framework web cho Node.js, xây dựng RESTful API
jsonwebtoken	9.0.x	Tạo và xác thực JWT token cho xác thực người dùng
mysql2	3.6.x	Kết nối và thao tác với cơ sở dữ liệu MySQL
multer	1.4.x	Middleware xử lý upload file/ảnh
@google/generative-ai	0.2.x	SDK tích hợp Google Gemini API
bcryptjs	2.4.x	Hash mật khẩu an toàn
dotenv	16.3.x	Quản lý biến môi trường từ file .env
cors	2.8.x	Xử lý Cross-Origin Resource Sharing
Frontend		
vue	3.3.x	Framework JavaScript progressive cho giao diện người dùng
vue-router	4.2.x	Quản lý routing cho Single Page Application
axios	1.5.x	HTTP client gửi request đến backend API
tailwindcss	3.3.x	Framework CSS utility-first cho styling nhanh
chart.js	4.4.x	Thư viện vẽ biểu đồ tương tác
vue-chartjs	5.2.x	Wrapper Chart.js cho Vue 3
Machine Learning		
tensorflow	2.14.x	Framework deep learning cho huấn luyện CNN
keras	2.14.x	API high-level cho xây dựng mô hình neural network
opencv-python	4.8.x	Xử lý ảnh và computer vision
numpy	1.24.x	Thư viện tính toán mảng và ma trận
scikit-learn	1.3.x	Đánh giá mô hình, confusion matrix, metrics

2.6.1 Backend: Node.js và Express.js

Node.js là môi trường chạy JavaScript phía server, sử dụng V8 engine của Chrome. Ưu điểm là xử lý bất đồng bộ hiệu quả, hệ sinh thái npm phong phú và cùng ngôn ngữ với frontend.

Express.js là framework web tối giản, linh hoạt cho Node.js. Hệ thống sử dụng kiến trúc RESTful API với các endpoint chuẩn: GET (lấy dữ liệu), POST (tạo mới), PUT (cập nhật), DELETE (xóa). Các route được tổ chức theo module: `/api/auth` (xác thực), `/api/classify` (phân loại), `/api/history` (lịch sử), `/api/statistics` (thống kê).

JWT Authentication: JSON Web Token được dùng để xác thực stateless. Sau khi đăng nhập thành công, server tạo JWT chứa thông tin user và gửi cho client. Client gửi kèm JWT trong header `Authorization: Bearer <token>` cho các request tiếp theo. Middleware xác thực JWT trước khi cho phép truy cập các route được bảo vệ [8].

MySQL với mysql2: Cơ sở dữ liệu quan hệ lưu trữ thông tin người dùng, lịch sử phân loại, thống kê. Thư viện `mysql2` cung cấp API Promise-based, hỗ trợ prepared statement bảo vệ khỏi SQL injection.

Multer upload: Middleware xử lý multipart/form-data để upload file ảnh. Hệ thống cấu hình giới hạn kích thước file (5MB), định dạng cho phép (JPEG, PNG) và lưu tạm vào thư mục `uploads/`.

2.6.2 Frontend: Vue.js 3

Vue.js 3 Composition API: Vue 3 sử dụng Composition API thay vì Options API, cho phép tổ chức logic theo chức năng thay vì loại option, tái sử dụng code tốt hơn với composables. Hệ thống dùng `<script setup>` syntax ngắn gọn và reactive state với `ref()`, `reactive()`.

Vue Router: Quản lý điều hướng client-side cho SPA. Các route chính: `/` (trang chủ), `/login` (đăng nhập), `/classify` (phân loại), `/history` (lịch sử), `/statistics` (thống kê). Navigation guard kiểm tra xác thực trước khi vào route yêu cầu đăng nhập.

Axios: HTTP client gửi request REST API đến backend. Cấu hình interceptor tự động thêm JWT token vào header và xử lý lỗi 401 (unauthorized) để chuyển về trang đăng nhập.

Tailwind CSS: Framework CSS utility-first cho phép styling nhanh bằng các

class tiện ích như `flex`, `text-center`, `bg-blue-500`. Ưu điểm là file CSS nhỏ nhờ tree-shaking, dễ responsive và custom theme.

2.6.3 Progressive Web App (PWA)

PWA là công nghệ cho phép web app hoạt động như native app với khả năng cài đặt, chạy ngoại tuyến và nhận thông báo đẩy.

Service Worker: Script chạy nền, độc lập với trang web, chặn request mạng và cache tài nguyên. Hệ thống dùng service worker để cache các file tĩnh (HTML, CSS, JS, ảnh icon) cho phép app mở nhanh và chạy ngoại tuyến cơ bản.

Web App Manifest: File JSON (`manifest.json`) mô tả metadata của app: tên, icon, màu chủ đạo, chế độ hiển thị. Trình duyệt di động sử dụng manifest để hiển thị banner cài đặt và tùy chỉnh giao diện khi chạy standalone.

Offline Support: Khi không có kết nối mạng, service worker trả về nội dung từ cache. Các chức năng cần API (phân loại, đồng bộ dữ liệu) thông báo người dùng và lưu hàng đợi để gửi lại khi có mạng.

2.7 Nghiên cứu liên quan

Nhiều nghiên cứu và hệ thống phân loại rác thải sử dụng AI đã được công bố. Bảng 2.3 tóm tắt một số hệ thống tiêu biểu.

Bảng 2.3. So sánh các hệ thống phân loại rác thải sử dụng AI.

Nghiên cứu	Phạm vi	Nội dung liên quan
Yang và Thung [15]	Bộ dữ liệu TrashNet	Cung cấp tập ảnh phân loại rác thường dùng trong các nghiên cứu thị giác máy tính.
Aral và cộng sự [2]	Học sâu trên TrashNet	Khảo sát các mô hình học sâu trên bộ dữ liệu TrashNet.
Adedeji và Wang [1]	Hệ thống phân loại rác bằng CNN	Trình bày một hệ thống phân loại rác sử dụng mạng nơ-ron tích chập.
Bircanoglu và cộng sự [3]	RecycleNet	Nghiên cứu phân loại rác bằng mạng nơ-ron sâu.
Lu và Chen [12]	Tổng quan thị giác máy tính	Tổng quan các nghiên cứu về phân loại chất thải rắn bằng thị giác máy tính.

Các tài liệu trên lần lượt cung cấp dữ liệu chuẩn để thử nghiệm, khảo sát mô hình học sâu và tổng quan hướng ứng dụng thị giác máy tính vào phân loại chất thải. Đồ án này tập trung vào một ứng dụng web tích hợp Gemini để phân tích ảnh, đồng thời bổ sung các chức năng tài khoản, lịch sử và tư vấn người dùng; kết quả thực nghiệm của đồ án được trình bày riêng ở Chương 5.

CHƯƠNG 3

Phân tích và thiết kế hệ thống

MỤC TIÊU CHƯƠNG

Chương này trình bày chi tiết quá trình phân tích yêu cầu, thiết kế kiến trúc hệ thống, thiết kế use case, thiết kế cơ sở dữ liệu, thiết kế API *RESTful* và thiết kế giao diện người dùng cho ứng dụng phân loại rác thải thông minh. Mục tiêu là xây dựng một hệ thống hoàn chỉnh sử dụng trí tuệ nhân tạo để hỗ trợ người dùng trong việc phân loại và tái chế rác thải đúng cách.

3.1 Phân tích yêu cầu

Phần này mô tả chi tiết các yêu cầu chức năng và phi chức năng của hệ thống, làm cơ sở cho các bước thiết kế và triển khai tiếp theo.

3.1.1 Yêu cầu chức năng

Bảng 3.1 liệt kê các yêu cầu chức năng chính của hệ thống. Mỗi yêu cầu được gán mã định danh, tên chức năng, mô tả chi tiết và mức độ ưu tiên trong quá trình triển khai.

Bảng 3.1. Danh sách yêu cầu chức năng của hệ thống.

STT	Mã	Tên chức năng	Độ ưu tiên
1	FR01	Đăng ký tài khoản với email, mật khẩu và username	Cao
2	FR02	Đăng nhập/Đăng xuất sử dụng JWT và mã hoá bcrypt	Cao
3	FR03	Upload ảnh rác và phân loại tự động bằng Gemini Vision	Cao
4	FR04	Xem kết quả phân loại với loại rác, độ tin cậy, hướng dẫn	Cao
5	FR05	Chatbot tư vấn AI sử dụng Gemini multi-turn	Trung bình
6	FR06	Xem lịch sử phân loại đã thực hiện	Trung bình
7	FR07	Quản lý thông tin, bảo mật và phản hồi cá nhân	Thấp
8	FR08	Cài đặt ứng dụng trên thiết bị di động	Thấp
9	FR09	Chat với nhân viên hỗ trợ	Trung bình
10	FR10	Tạo và theo dõi đơn thu mua rác	Trung bình
11	FR11	Xem giá thu mua và thông tin cửa hàng	Trung bình
12	FR12	Quản trị người dùng, nội dung và hoạt động hệ thống	Cao

Chức năng FR03 là trung tâm của hệ thống, cho phép người dùng chụp hoặc tải lên ảnh rác thải, sau đó hệ thống sử dụng API *Gemini Vision* của Google để nhận diện và phân loại rác thải. Chức năng FR05 cung cấp tư vấn tự động; FR09 bổ sung kênh trao đổi với nhân viên. Các chức năng FR10–FR11 hỗ trợ tạo đơn thu mua và tra cứu giá, còn FR12 dành cho quản trị người dùng, nhân viên, nội dung, thống kê và các hoạt động liên quan. Chi tiết trạng thái và quy trình xử lý các chức năng mở rộng cần được đối chiếu với mã nguồn triển khai.

3.1.2 Yêu cầu phi chức năng

Bảng 3.2 mô tả các yêu cầu phi chức năng về hiệu năng, độ chính xác, bảo mật, khả dụng và tương thích của hệ thống.

Bảng 3.2. Danh sách yêu cầu phi chức năng của hệ thống.

Loại yêu cầu	Mô tả chi tiết
Hiệu năng	Thời gian phản hồi API phân loại ảnh dưới 3 giây; thời gian load trang dưới 2 giây trên kết nối 3G
Độ chính xác	Đạt độ chính xác tối thiểu 85% trên tập ảnh rác thải phổ biến ở Việt Nam
Bảo mật	Mật khẩu được mã hoá bằng bcrypt với 10 rounds; JWT token hết hạn sau 24 giờ; rate limiting 100 requests mỗi 15 phút mỗi IP
Khả dụng	Giao diện responsive trên cả desktop và mobile; hỗ trợ PWA với khả năng cache nội dung tĩnh offline
Tương thích	Hỗ trợ các trình duyệt Chrome, Firefox, Safari, Edge phiên bản mới nhất trong 2 năm

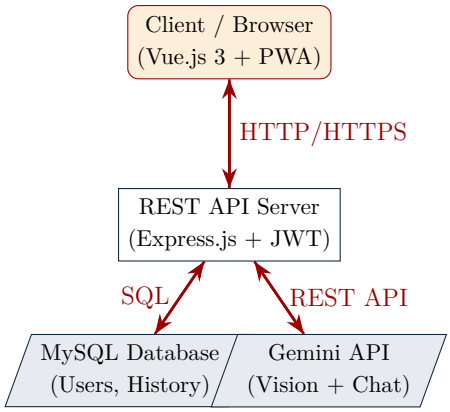
Yêu cầu về bảo mật đảm bảo dữ liệu người dùng được bảo vệ thông qua mã hoá mật khẩu mạnh và cơ chế xác thực token. Yêu cầu về hiệu năng đảm bảo trải nghiệm người dùng mượt mà ngay cả trên mạng di động có băng thông hạn chế.

3.2 Kiến trúc hệ thống

Hệ thống được thiết kế theo kiến trúc ba tầng *(three-tier architecture)* gồm tầng trình diễn, tầng xử lý logic và tầng dữ liệu, với sự tích hợp của dịch vụ AI bên ngoài.

3.2.1 Kiến trúc tổng quan

Hình 3.1 mô tả kiến trúc tổng quan của hệ thống với ba tầng chính và luồng dữ liệu giữa các thành phần.



Hình 3.1. Sơ đồ kiến trúc tổng thể ba tầng của hệ thống.

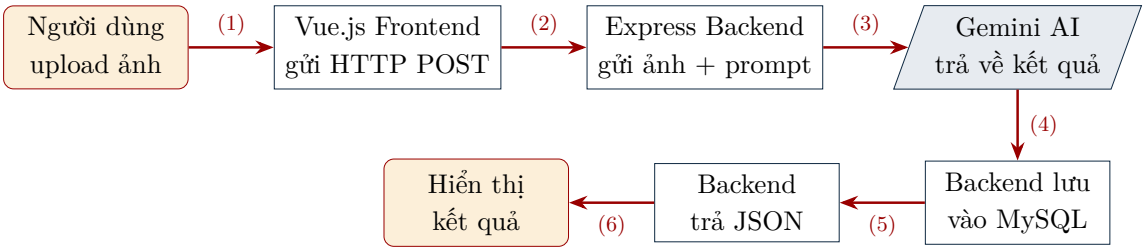
Tầng trình diễn sử dụng framework *Vue.js 3* với các tính năng *Progressive Web App* (PWA) để cung cấp giao diện người dùng tương tác. Tầng xử lý logic sử dụng *Express.js* để xây dựng API *RESTful*, xử lý xác thực và phân quyền thông qua JWT. Tầng dữ liệu sử dụng cơ sở dữ liệu quan hệ MySQL để lưu trữ thông tin người dùng, lịch sử phân loại và lịch sử trò chuyện. Hệ thống tích hợp với API *Gemini* của Google để thực hiện phân loại ảnh và trò chuyện tư vấn.

3.2.2 Luồng dữ liệu phân loại ảnh

Quá trình phân loại ảnh rác thải được thực hiện theo các bước sau:

1. Người dùng đã đăng nhập chọn hoặc chụp ảnh rác thải từ giao diện web.
2. Ảnh được upload lên server thông qua API `POST /api/classify` dưới dạng *multipart/form-data*.
3. Middleware *multer* trên server nhận và lưu ảnh vào buffer bộ nhớ, không lưu xuống đĩa để tăng hiệu năng.
4. Server chuyển đổi buffer ảnh sang định dạng *base64* và gửi đến API *Gemini Vision* kèm prompt yêu cầu phân loại vào 8 loại rác.
5. API *Gemini* trả về kết quả JSON chứa loại rác đã phân loại, độ tin cậy *(confidence score)* và hướng dẫn tái chế.
6. Server lưu kết quả vào bảng `classification_history` trong MySQL với user ID, mã hash ảnh và thông tin phân loại.
7. Kết quả được trả về cho client dưới dạng JSON để hiển thị cho người dùng.

Hình 3.2 mô tả luồng này bằng sơ đồ trực quan với các bước được đánh số từ 1 đến 7.



Hình 3.2. Sơ đồ luồng hệ thống phân loại rác thải từ người dùng đến kết quả hiển thị.

3.2.3 Kiến trúc frontend Vue.js 3

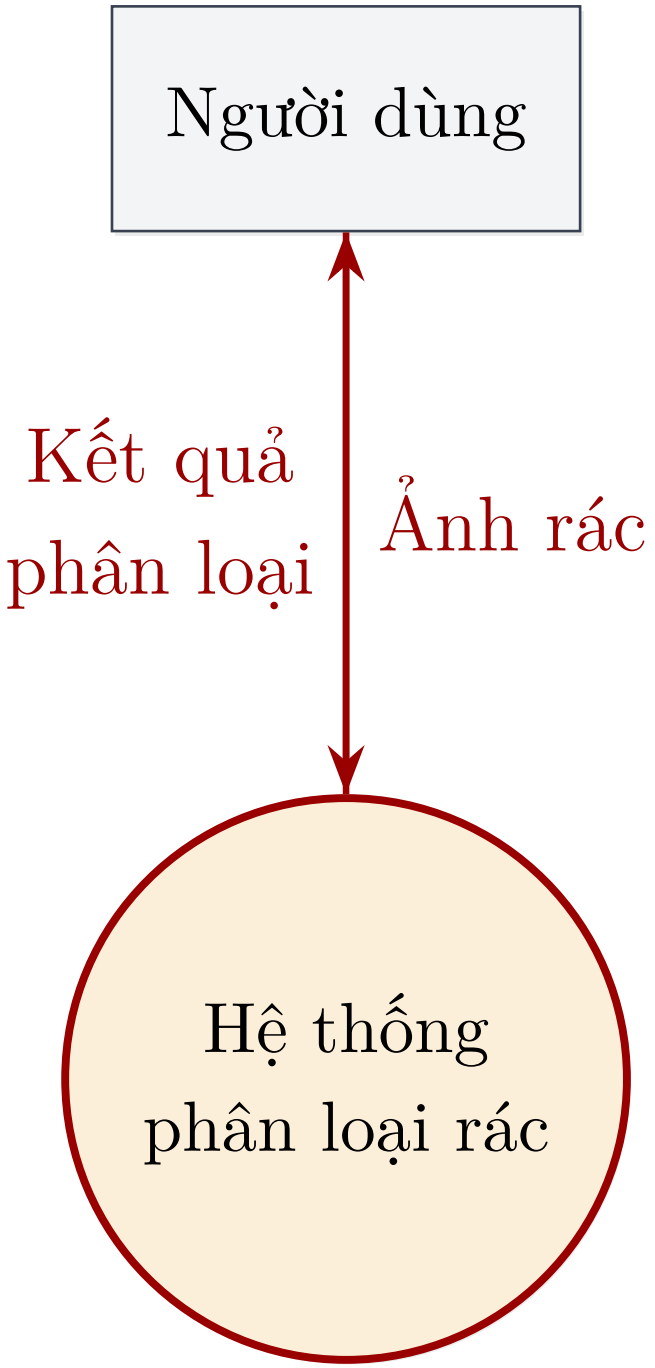
Frontend được tổ chức theo cấu trúc component-based với các thành phần chính:

- **Components:** các component tái sử dụng như `ClassifyCard`, `ChatMessage`, `HistoryItem`, `WasteTypeIcon`.
- **Views:** các trang chính tương ứng với route như `HomeView`, `ClassifyView`, `ChatView`, `HistoryView`.
- **Router:** quản lý điều hướng giữa các trang với *Vue Router*, bảo vệ các route yêu cầu xác thực.
- **Composables:** các hàm logic tái sử dụng như `useAuth` (quản lý xác thực), `useApi` (gọi API), `useToast` (hiển thị thông báo).
- **Stores:** quản lý trạng thái toàn cục với *Pinia* cho thông tin user và waste types.

3.2.4 Sơ đồ luồng dữ liệu (DFD)

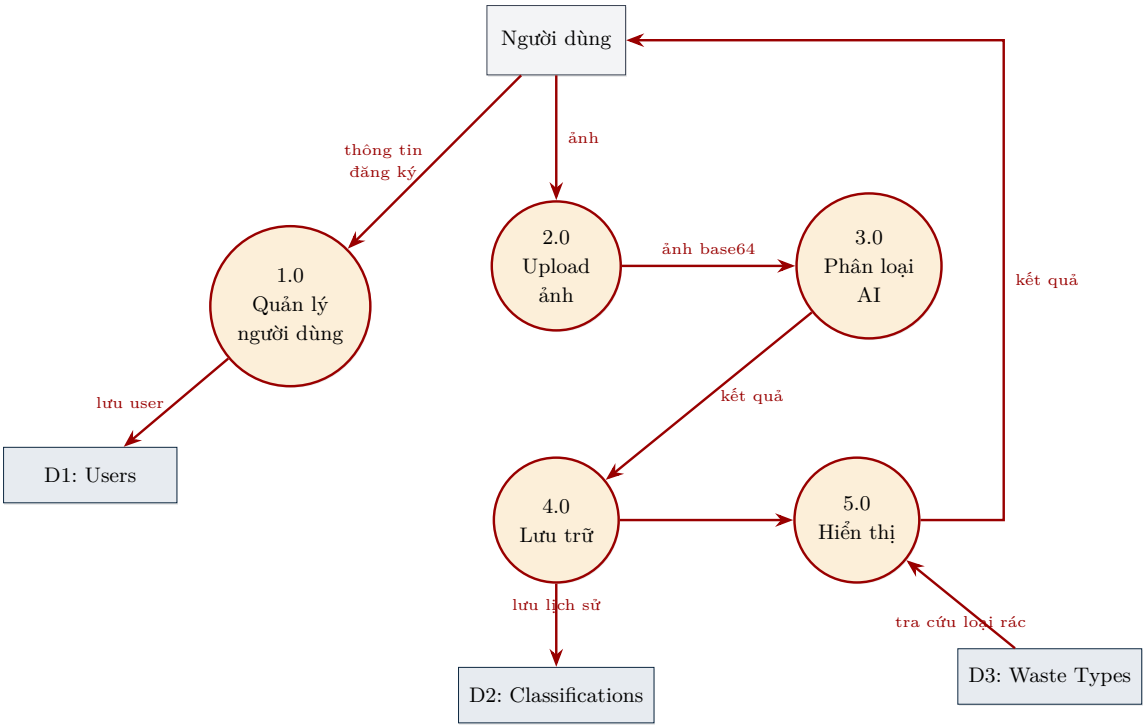
Sơ đồ luồng dữ liệu (DFD) mô tả cách dữ liệu di chuyển qua các thành phần của hệ thống, từ mức độ tổng quan (Level 0) đến chi tiết (Level 2).

Hình 3.3 mô tả biểu đồ ngữ cảnh với người dùng và hệ thống.



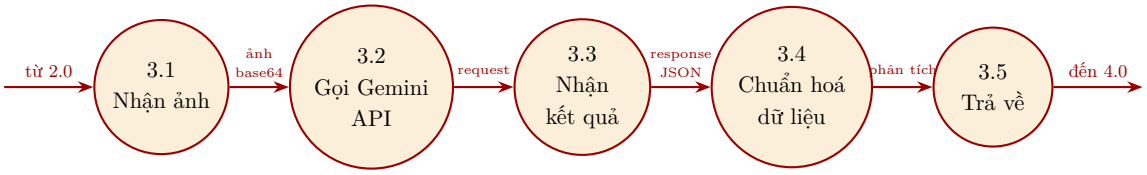
Hình 3.3. DFD Level 0: Biểu đồ ngữ cảnh của hệ thống phân loại rác thải.

Hình 3.4 mô tả sự phân rã của hệ thống thành 5 process và 3 datastore.



Hình 3.4. DFD Level 1: Phân rã hệ thống thành 5 process và 3 datastore.

Hình 3.5 mô tả chi tiết process 3.0 Phân loại AI thành 5 sub-process.



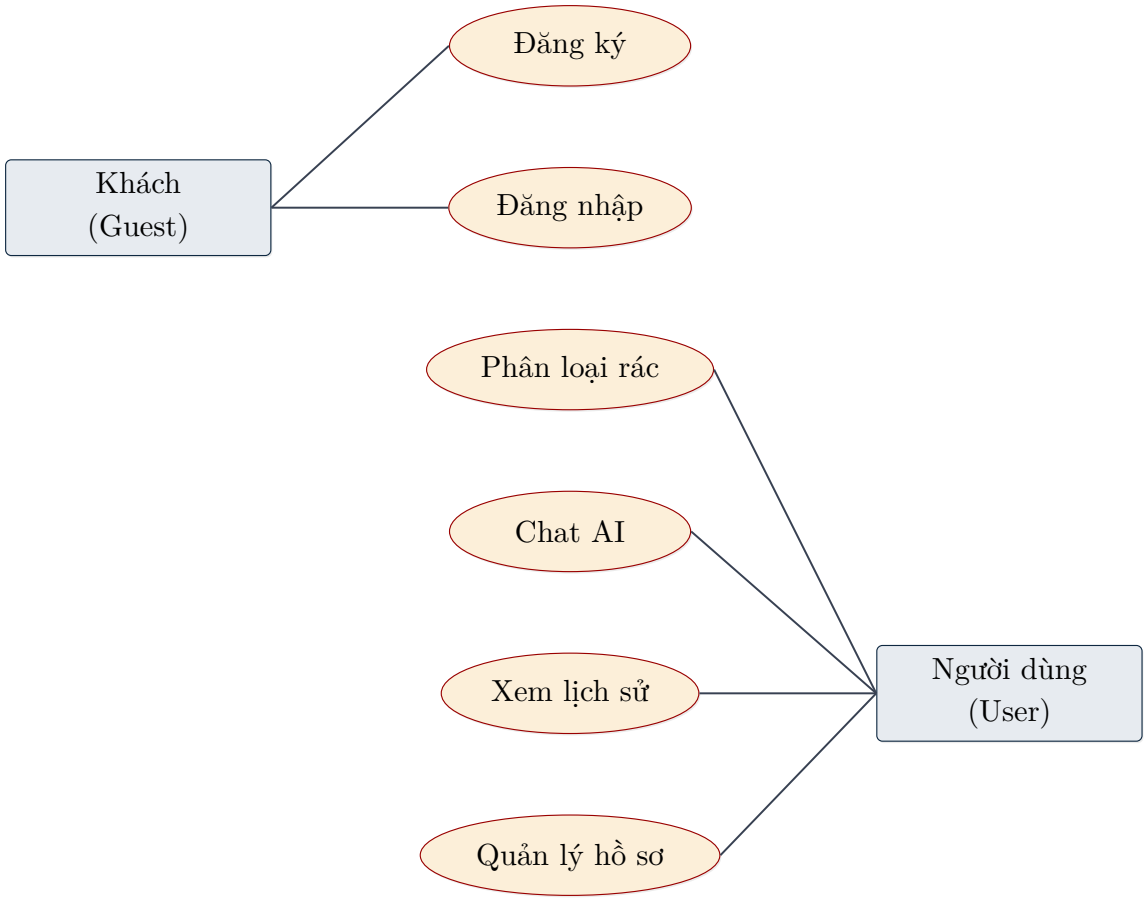
Hình 3.5. DFD Level 2: Chi tiết process 3.0 Phân loại AI thành 5 sub-process.

3.3 Thiết kế Use Case

Phần này mô tả các tác nhân và các trường hợp sử dụng chính của hệ thống.

3.3.1 Sơ đồ use case

Hình 3.6 mô tả hai actor và sáu use case chính của hệ thống.



Hình 3.6. Sơ đồ use case với hai actor và sáu chức năng chính của hệ thống.

3.3.2 Danh sách actor và use case

Hệ thống có hai loại tác nhân chính:

- **Khách (Guest):** người dùng chưa đăng nhập, có thể xem trang chủ, đăng ký tài khoản và đăng nhập.
- **Người dùng đăng nhập (User):** người dùng đã xác thực, có quyền sử dụng đầy đủ các chức năng phân loại rác, chat tư vấn, xem lịch sử và quản lý thông tin cá nhân.

3.3.3 Mô tả chi tiết use case

Các bảng sau mô tả chi tiết bốn use case chính của hệ thống.

Bảng 3.3. Use case UC01: Đăng ký tài khoản.

Thuộc tính	Mô tả
Mã UC	UC01
Tên	Đăng ký tài khoản
Actor	Khách (Guest)
Precondition	Người dùng chưa có tài khoản trong hệ thống
Main flow	1. Khách truy cập trang đăng ký 2. Nhập username, email và mật khẩu 3. Hệ thống kiểm tra tính hợp lệ (email chưa tồn tại, mật khẩu đủ mạnh) 4. Hệ thống mã hoá mật khẩu bằng bcrypt 5. Tạo bản ghi mới trong bảng users 6. Trả về JWT token và thông tin người dùng
Exception	Email đã tồn tại: hiển thị lỗi và yêu cầu nhập email khác Mật khẩu không đủ mạnh: hiển thị yêu cầu về độ dài và độ phức tạp
Postcondition	Tài khoản mới được tạo, người dùng đăng nhập tự động

Bảng 3.4. Use case UC02: Phân loại rác thải.

Thuộc tính	Mô tả
Mã UC	UC02
Tên	Phân loại rác thải
Actor	Người dùng đăng nhập (User)
Precondition	Người dùng đã đăng nhập và có token JWT hợp lệ
Main flow	1. Người dùng truy cập trang phân loại 2. Chọn hoặc chụp ảnh rác thải 3. Nhấn nút phân loại, ảnh được upload lên server 4. Server gửi ảnh đến Gemini Vision API 5. Gemini phân loại rác vào một trong 8 loại: organic, recyclable, paper, plastic, glass, metal, hazardous, e-waste 6. Kết quả trả về bao gồm loại rác, confidence score, hướng dẫn tái chế 7. Server lưu kết quả vào lịch sử 8. Hiển thị kết quả với màu sắc và icon tương ứng
Exception	Ảnh không hợp lệ (quá lớn, sai định dạng): hiển thị lỗi và yêu cầu chọn ảnh khác API Gemini lỗi: hiển thị thông báo lỗi và yêu cầu thử lại
Postcondition	Kết quả phân loại được lưu vào lịch sử người dùng

Bảng 3.5. Use case UC03: Chat tư vấn AI.

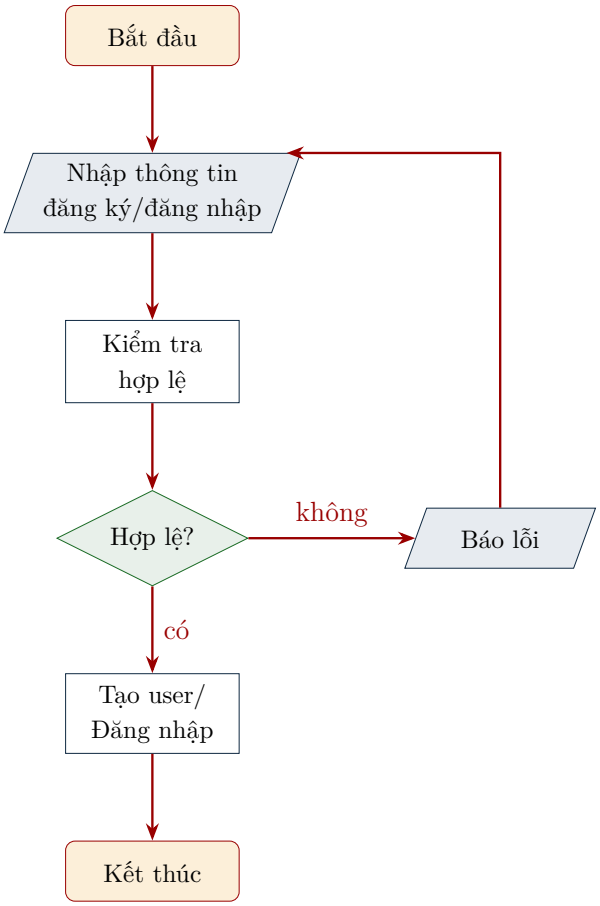
Thuộc tính	Mô tả
Mã UC	UC03
Tên	Chat tư vấn AI về rác thải và tái chế
Actor	Người dùng đăng nhập (User)
Precondition	Người dùng đã đăng nhập
Main flow	1. Người dùng truy cập trang chat 2. Nhập câu hỏi về phân loại hoặc tái chế rác 3. Server gửi câu hỏi đến Gemini API với context về hệ thống phân loại rác 4. Gemini trả về câu trả lời phù hợp 5. Server lưu câu hỏi và trả lời vào bảng chat_history 6. Hiển thị câu trả lời cho người dùng 7. Người dùng có thể tiếp tục hỏi (multi-turn conversation)
Exception	API Gemini không phản hồi: hiển thị thông báo lỗi
Postcondition	Lịch sử trò chuyện được lưu lại để tham khảo sau

Bảng 3.6. Use case UC04: Xem lịch sử phân loại.

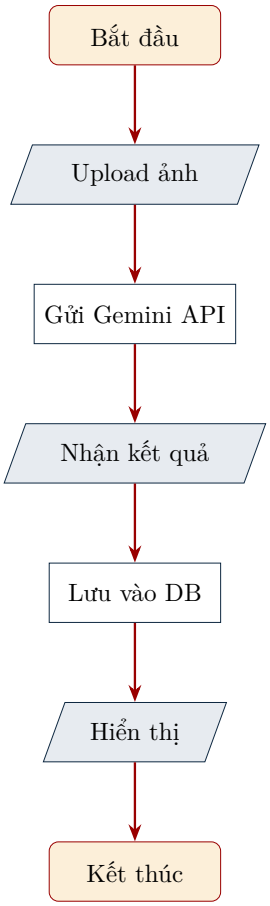
Thuộc tính	Mô tả
Mã UC	UC04
Tên	Xem lịch sử phân loại
Actor	Người dùng đăng nhập (User)
Precondition	Người dùng đã đăng nhập
Main flow	1. Người dùng truy cập trang lịch sử 2. Server truy vấn bảng classification_history theo user_id 3. Trả về danh sách các lần phân loại đã thực hiện 4. Hiển thị danh sách với thông tin: loại rác, độ tin cậy, thời gian, icon màu sắc 5. Người dùng có thể xem chi tiết từng kết quả
Exception	Không có lịch sử: hiển thị thông báo và gợi ý thử phân loại ảnh
Postcondition	Người dùng xem được lịch sử phân loại của mình

3.3.4 Sơ đồ hoạt động

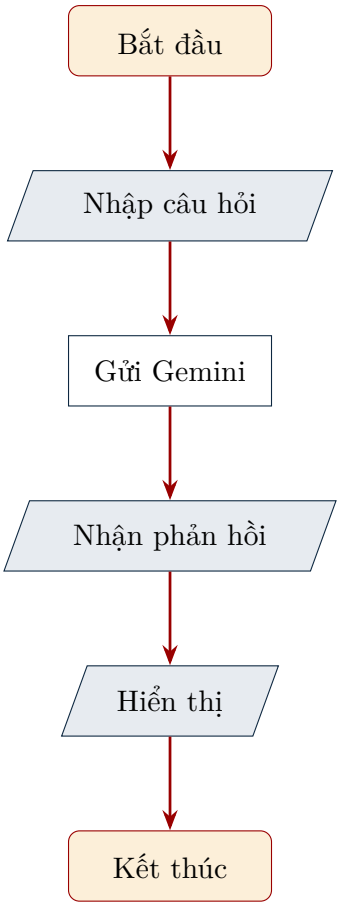
Ba sơ đồ hoạt động sau mô tả chi tiết luồng xử lý của các chức năng chính: đăng ký/đăng nhập, phân loại rác và chat tư vấn AI. Hình 3.7, 3.8 và 3.9 mô tả chi tiết luồng xử lý từng chức năng.



Hình 3.7. Sơ đồ hoạt động của quy trình đăng ký và đăng nhập.



Hình 3.8. Sơ đồ hoạt động của quy trình phân loại rác thải.



Hình 3.9. Sơ đồ hoạt động của quy trình chat tư vấn với AI.

3.4 Thiết kế cơ sở dữ liệu

Hệ thống sử dụng cơ sở dữ liệu quan hệ MySQL với bốn bảng chính để lưu trữ dữ liệu người dùng, loại rác, lịch sử phân loại và lịch sử trò chuyện.

3.4.1 Mô hình quan hệ

Cơ sở dữ liệu gồm bốn bảng với các mối quan hệ sau:

- **users:** lưu thông tin tài khoản người dùng bao gồm id (khoá chính, tự động tăng), username (VARCHAR 50, duy nhất), email (VARCHAR 100, duy nhất), password_hash (VARCHAR 255 chứa mật khẩu đã mã hoá bcrypt), created_at (timestamp thời gian tạo tài khoản).
- **waste_types:** lưu danh sách 8 loại rác với id (khoá chính), code (VARCHAR 30, duy nhất), name_vi (VARCHAR 50, tên tiếng Việt), name_en (VARCHAR 50, tên tiếng Anh), color (VARCHAR 7, mã màu hex), icon (VARCHAR 10, tên icon), recycling_guide (TEXT, hướng dẫn tái chế).
- **classification_history:** lưu lịch sử phân loại với id (khoá chính), user_id (khoá ngoại tham chiếu users.id), image_hash (VARCHAR 32, mã hash MD5 của ảnh), waste_type_code (VARCHAR 30, mã loại rác), confidence (DECIMAL 5,2, độ tin cậy từ 0 đến 100), raw_result (TEXT, kết quả JSON gốc từ Gemini), created_at (timestamp).
- **chat_history:** lưu lịch sử trò chuyện với id (khoá chính), user_id (khoá ngoại tham chiếu users.id), message (TEXT, câu hỏi người dùng), response (TEXT, câu trả lời từ AI), created_at (timestamp).

3.4.2 Bảng mô tả chi tiết

Các bảng sau mô tả chi tiết cấu trúc từng bảng trong cơ sở dữ liệu.

Bảng 3.7. Cấu trúc bảng users.

Trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	INT	PK, tự tăng	Mã định danh người dùng
username	VARCHAR(50)	UNIQUE, NOT NULL	Tên đăng nhập
email	VARCHAR(100)	UNIQUE, NOT NULL	Email người dùng
password_hash	VARCHAR(255)	NOT NULL	Mật khẩu đã mã hoá bcrypt
created_at	TIMESTAMP	Mặc định thời gian hiện tại	Thời gian tạo tài khoản

Bảng 3.8. Cấu trúc bảng waste_types.

Trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	INT	PK, tự tăng	Mã định danh loại rác
code	VARCHAR(30)	UNIQUE, NOT NULL	Mã loại rác (organic, plastic, ...)
name_vi	VARCHAR(50)	NOT NULL	Tên tiếng Việt
name_en	VARCHAR(50)	NOT NULL	Tên tiếng Anh
color	VARCHAR(7)	NOT NULL	Mã màu hex (ví dụ #22c55e)
icon	VARCHAR(10)	NOT NULL	Tên icon hiển thị
recycling_guide	TEXT	NULL	Hướng dẫn tái chế chi tiết

Bảng 3.9. Cấu trúc bảng classification_history.

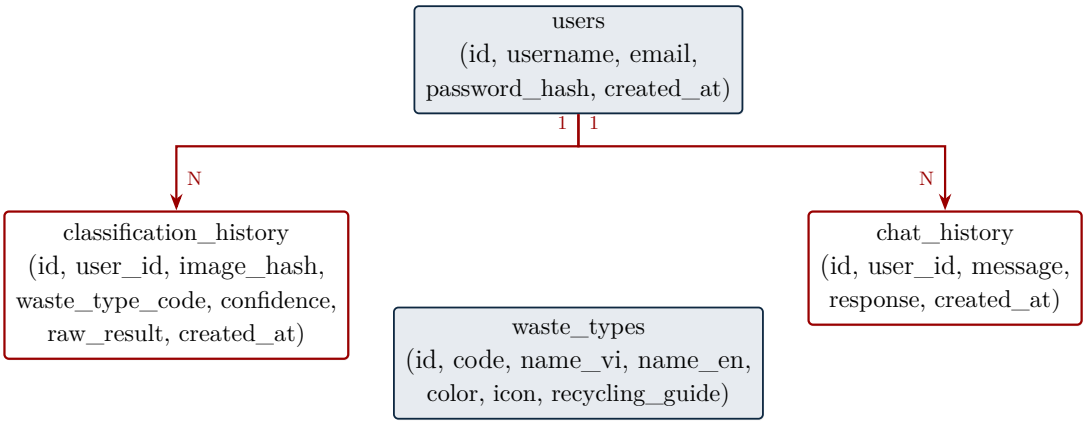
Trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	INT	PK, tự tăng	Mã định danh bản ghi
user_id	INT	FK(users.id), NOT NULL	Người dùng thực hiện phân loại
image_hash	VARCHAR(32)	NOT NULL	Hash MD5 của ảnh
waste_type_code	VARCHAR(30)	NOT NULL	Loại rác phân loại được
confidence	DECIMAL(5,2)	NOT NULL	Độ tin cậy (0–100)
raw_result	TEXT	NULL	Kết quả JSON gốc từ Gemini
created_at	TIMESTAMP	Mặc định thời gian hiện tại	Thời gian phân loại

Bảng 3.10. Cấu trúc bảng chat_history.

Trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	INT	PK, tự tăng	Mã định danh bản ghi
user_id	INT	FK(users.id), NOT NULL	Người dùng chat
message	TEXT	NOT NULL	Tin nhắn từ người dùng
response	TEXT	NOT NULL	Phản hồi từ AI
created_at	TIMESTAMP	Mặc định thời gian hiện tại	Thời gian chat

3.4.3 Sơ đồ quan hệ

Hình 3.10 mô tả mối quan hệ giữa các bảng trong cơ sở dữ liệu.



Hình 3.10. Sơ đồ quan hệ giữa các bảng trong cơ sở dữ liệu.

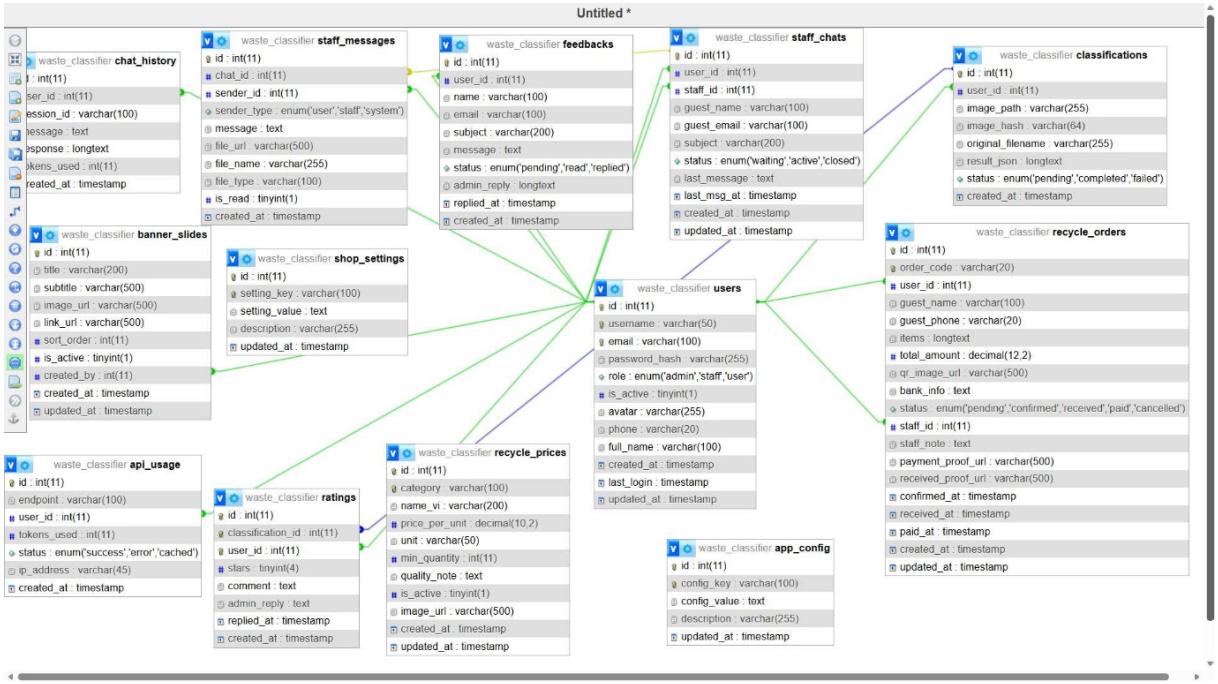
Bảng `users` có mối quan hệ một-nhiều với cả `classification_history` và `chat_history`, mỗi người dùng có thể có nhiều bản ghi lịch sử phân loại và lịch sử trò chuyện. Bảng `waste_types` là bảng tra cứu độc lập không có ràng buộc khoá ngoại, được nạp sẵn 8 loại rác khi khởi tạo hệ thống.

3.4.4 Ảnh chụp cơ sở dữ liệu triển khai

Mô hình phía trên trình bày bốn bảng lõi theo thiết kế khái niệm. Ảnh chụp schema triển khai ghi nhận 13 bảng trong cơ sở dữ liệu `waste_classifier`, trong đó một số tên bảng và cấu trúc khác với mô hình khái niệm. Các nhóm dữ liệu mở rộng phục vụ thống kê API, đánh giá, hỗ trợ khách hàng, quản trị nội dung và thu mua rác.

	Table	Action	Rows	Type	Collation	Size	Overhead
☐	api_usage	Browse Structure Search Insert Empty Drop	84	InnoDB	utf8mb4_unicode_ci	64.0 KiB	-
☐	app_config	Browse Structure Search Insert Empty Drop	7	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
☐	banner_slides	Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_unicode_ci	64.0 KiB	-
☐	chat_history	Browse Structure Search Insert Empty Drop	11	InnoDB	utf8mb4_unicode_ci	112.0 KiB	-
☐	classifications	Browse Structure Search Insert Empty Drop	53	InnoDB	utf8mb4_unicode_ci	208.0 KiB	-
☐	feedbacks	Browse Structure Search Insert Empty Drop	1	InnoDB	utf8mb4_unicode_ci	64.0 KiB	-
☐	ratings	Browse Structure Search Insert Empty Drop	8	InnoDB	utf8mb4_unicode_ci	96.0 KiB	-
☐	recycle_orders	Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_general_ci	96.0 KiB	-
☐	recycle_prices	Browse Structure Search Insert Empty Drop	5	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐	shop_settings	Browse Structure Search Insert Empty Drop	4	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐	staff_chats	Browse Structure Search Insert Empty Drop	19	InnoDB	utf8mb4_unicode_ci	80.0 KiB	-
☐	staff_messages	Browse Structure Search Insert Empty Drop	32	InnoDB	utf8mb4_unicode_ci	80.0 KiB	-
☐	users	Browse Structure Search Insert Empty Drop	10	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
	13 tables	Sum	239	InnoDB	utf8mb4_unicode_ci	1.0 MiB	0 B

Hình 3.11. Danh sách 13 bảng của cơ sở dữ liệu triển khai trên phpMyAdmin.



Hình 3.12. Sơ đồ quan hệ cơ sở dữ liệu triển khai.

3.4.5 Các nhóm bảng trong schema triển khai

Bảng 3.11 tóm tắt các nhóm bảng thể hiện trong ảnh chụp schema.

Bảng 3.11. Nhóm bảng của cơ sở dữ liệu triển khai.

Nhóm	Bảng	Vai trò
Nghiệp vụ cốt lõi	`users`, `classifications`, `chat_history`	Tài khoản, kết quả phân loại và hội thoại AI.
Theo dõi và đánh giá	`api_usage`, `ratings`	Ghi nhận lượt dùng API và đánh giá kết quả phân loại.
Hỗ trợ người dùng	`staff_chats`, `staff_messages`, `feedbacks`	Hội thoại với nhân viên, tin nhắn và phản hồi.
Thu mua rác	`recycle_orders`, `recycle_prices`, `shop_settings`	Đơn hàng, bảng giá và thông tin cửa hàng.
Quản trị nội dung	`app_config`, `banner_slides`	Cấu hình ứng dụng và banner hiển thị.

Sơ đồ quan hệ còn thể hiện các liên kết giữa tài khoản với hoạt động phân loại, đơn thu mua và phiên hỗ trợ; đánh giá gắn với kết quả phân loại, còn tin nhắn được tổ chức theo phiên hỗ trợ. Đây là mô tả schema tại thời điểm chụp ảnh.

3.5 Thiết kế API RESTful

Hệ thống cung cấp các API endpoint tuân thủ nguyên tắc *RESTful* để frontend giao tiếp với backend. Bảng 3.12 liệt kê chi tiết các endpoint.

Bảng 3.12. Danh sách API endpoints của hệ thống.

Method	Endpoint	Mô tả	Auth
POST	/api/auth/register	Đăng ký tài khoản mới	Không
POST	/api/auth/login	Đăng nhập và nhận JWT token	Không
GET	/api/auth/profile	Lấy thông tin người dùng hiện tại	JWT
POST	/api/classify	Upload và phân loại ảnh rác	JWT
GET	/api/history	Xem lịch sử phân loại	JWT
GET	/api/waste-types	Lấy danh sách 8 loại rác	Không
POST	/api/chat	Gửi tin nhắn và nhận phản hồi AI	JWT
GET	/api/chat/history	Xem lịch sử trò chuyện	JWT

Chi tiết request và response:

- `POST /api/auth/register`: Request body `{username,email,password}`; Response `{token,user: {id,username,email}}`.
- `POST /api/auth/login`: Request body `{email,password}`; Response `{token, user: {id,username,email}}`.
- `GET /api/auth/profile`: Header `Authorization`; Response `{user: {id, username,email,created_at}}`.
- `POST /api/classify`: Content-Type `multipart/form-data`, field `image` (file); Response `{waste_type,confidence,guide,color,icon}`.
- `GET /api/history`: Response `[{id,waste_type,confidence,created_at}]`.
- `GET /api/waste-types`: Response `[{code,name_vi,name_en,color,icon}]`.
- `POST /api/chat`: Request body `{message}`; Response `{response}`.
- `GET /api/chat/history`: Response `[{message,response,created_at}]`.

Tất cả các endpoint yêu cầu xác thực đều kiểm tra JWT token trong header `Authorization`. Token hết hạn sau 24 giờ và cần đăng nhập lại. Hệ thống áp dụng rate limiting 100 requests mỗi 15 phút cho mỗi địa chỉ IP để chống spam và tấn công.

3.6 Thiết kế giao diện người dùng

Giao diện người dùng được thiết kế theo nguyên tắc *responsive-first*, ưu tiên trải nghiệm trên thiết bị di động và sử dụng hệ màu trực quan để phân biệt các loại rác.

3.6.1 Màu sắc và thương hiệu

Hệ thống sử dụng bảng màu dựa trên 8 loại rác, mỗi loại được gán một màu riêng biệt để người dùng dễ dàng nhận diện:

- **Organic** (rác hữu cơ): màu xanh lá `#22c55e`
- **Recyclable** (rác tái chế): màu xanh dương `#3b82f6`
- **Paper** (giấy): màu vàng `#eab308`
- **Plastic** (nhựa): màu cam `#f97316`
- **Glass** (thủy tinh): màu cyan `#06b6d4`
- **Metal** (kim loại): màu xám `#6b7280`
- **Hazardous** (nguy hại): màu đỏ `#ef4444`
- **E-waste** (điện tử): màu tím `#a855f7`

Hệ thống kết hợp framework *Tailwind CSS* để xây dựng giao diện responsive nhanh chóng với các utility classes có sẵn.

3.6.2 Danh sách các màn hình

Bảng 3.13 mô tả các màn hình chính của ứng dụng.

Bảng 3.13. Danh sách các màn hình trong ứng dụng.

Tên màn hình	Route	Mô tả chức năng
Home	/	Trang chủ giới thiệu hệ thống, danh sách 8 loại rác với icon và màu sắc
Login	/login	Form đăng nhập với email và mật khẩu
Register	/register	Form đăng ký tài khoản với username, email, password
Classify	/classify	Upload hoặc chụp ảnh rác, nút phân loại, hiển thị trạng thái loading
Result	/result	Hiển thị kết quả phân loại: loại rác, confidence score, hướng dẫn tái chế, màu nền tương ứng
Chat	/chat	Giao diện chat với AI, danh sách tin nhắn và ô nhập liệu
History	/history	Danh sách lịch sử phân loại theo thời gian, mỗi item hiển thị loại rác và confidence
Profile	/profile	Thông tin cá nhân người dùng, nút đổi mật khẩu và đăng xuất

3.6.3 Nguyên tắc thiết kế UX

Hệ thống tuân thủ các nguyên tắc thiết kế trải nghiệm người dùng sau:

- **Responsive-first:** sử dụng *Tailwind CSS* với các breakpoint `sm`, `md`, `lg`, `xl` để đảm bảo giao diện tối ưu trên mọi kích thước màn hình từ mobile đến desktop.
- **Phản hồi trực quan:** hiển thị trạng thái loading khi gọi API, sử dụng toast notification để thông báo thành công hoặc lỗi, skeleton loading cho danh sách lịch sử.
- **Màu sắc trực quan:** mỗi loại rác có màu riêng biệt, kết quả phân loại hiển thị với màu nền tương ứng để người dùng nhận diện nhanh.
- **Hỗ trợ PWA:** cấu hình *service worker* để cache các file tĩnh (HTML, CSS,

JS, icon), cho phép người dùng cài đặt ứng dụng lên màn hình chính thiết bị, hỗ trợ làm việc offline hạn chế với các trang đã cache.

- **Accessibility:** sử dụng semantic HTML, các form có label rõ ràng, contrast ratio màu sắc đạt chuẩn WCAG AA, hỗ trợ điều hướng bằng bàn phím.

CHƯƠNG 4

Cài đặt hệ thống

MỤC TIÊU CHƯƠNG

Chương này trình bày chi tiết quá trình cài đặt hệ thống phân loại rác thải thông minh, bao gồm môi trường phát triển, cấu trúc và triển khai các thành phần *backend*, *frontend*, cấu hình cơ sở dữ liệu, và quy trình triển khai lên môi trường thực tế. Các đoạn mã nguồn quan trọng được minh họa để làm rõ cách thức tích hợp các công nghệ *AI*, xác thực người dùng, xử lý tải lên ảnh và giao tiếp giữa các tầng ứng dụng.

4.1 Môi trường phát triển

Bảng 4.1 liệt kê các công cụ và phiên bản phần mềm được sử dụng trong quá trình phát triển hệ thống. Môi trường này đảm bảo tính tương thích và ổn định cho cả giai đoạn phát triển và triển khai.

Bảng 4.1. Môi trường phát triển hệ thống

Công cụ / Thư viện	Phiên bản
Node.js	v18 LTS
npm	v10
MySQL	8.0
Visual Studio Code	1.90
Git	2.45
Postman	(phiên bản mới nhất)
Overleaf	(dùng viết báo cáo)

Hệ điều hành sử dụng trong quá trình phát triển là Windows 11, với cấu hình phần cứng CPU Intel Core i7, RAM 16 GB. Mã nguồn được quản lý bằng *Git* và lưu trữ trên kho riêng tư.

4.2 Cài đặt backend

4.2.1 Cấu trúc thư mục backend

Cấu trúc thư mục *backend* được tổ chức theo mô hình *MVC* (*Model-View-Controller*) để dễ dàng bảo trì và mở rộng. Mã nguồn 4.1 mô tả cấu trúc thư mục chi tiết.

Mã nguồn 4.1. Cấu trúc thư mục backend

```
backend/
|-- src/
|   |-- config/
|   |   |-- database.js       # ếKt ốni MySQL
|   |   `-- gemini.js         # ởKhi ạto Gemini AI
|   |-- middleware/
|   |   |-- auth.js           # Xác ựthc JWT
|   |   `-- upload.js         # Multer upload ảnh
|   |-- routes/
|   |   |-- authRoutes.js     # /api/auth/*
|   |   |-- classifyRoutes.js # /api/classify
|   |   `-- chatRoutes.js     # /api/chat
|   |-- controllers/
|   |-- models/
|   `-- app.js
|-- .env
`-- package.json
```

4.2.2 Kết nối cơ sở dữ liệu MySQL

Hệ thống sử dụng thư viện `mysql2/promise` để kết nối đến cơ sở dữ liệu MySQL thông qua cơ chế *connection pool*, giúp tối ưu hóa hiệu suất khi có nhiều yêu cầu đồng thời. Mã nguồn 4.2 minh họa cách khởi tạo kết nối và xử lý lỗi.

Mã nguồn 4.2. Kết nối MySQL với connection pool

```
// src/config/database.js
const mysql = require('mysql2/promise');

// ạTo pool ếkt ốni để ốti ưu ệhiu ấsut
const pool = mysql.createPool({
  host: process.env.DB_HOST || 'localhost',
  port: process.env.DB_PORT || 3306,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  waitForConnections: true,
  connectionLimit: 10,
```

```
  queueLimit: 0
});

// ểKim tra ếkt ối khi ởkhi động
pool.getConnection()
  .then(connection => {
    console.log('ếKt ối MySQL thành công');
    connection.release();
  })
  .catch(err => {
    console.error('ỗLi ếkt ối MySQL:', err.message);
    process.exit(1);
  });

module.exports = pool;
```

4.2.3 Tích hợp Google Gemini AI

Google Gemini AI được sử dụng làm công cụ phân loại rác thải từ ảnh. Hệ thống khởi tạo mô hình `gemini-1.5-flash` và cung cấp hai chức năng chính: phân loại ảnh và *chatbot* tư vấn về tái chế.

4.2.3.1 Khởi tạo Gemini AI

Mã nguồn 4.3 mô tả cách khởi tạo *Google Generative AI* với *API key* từ biến môi trường.

Mã nguồn 4.3. Khởi tạo Google Gemini AI

```
// src/config/gemini.js
const { GoogleGenerativeAI } = require('@google/generative-ai');

// ởKhi ạto Gemini AI ới API key ừt .env
const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);

// ửS ụdng model gemini-1.5-flash cho ốtc độ ửx lý nhanh
const model = genAI.getGenerativeModel({
  model: 'gemini-1.5-flash'
});

module.exports = { model, genAI };
```

4.2.3.2 Phân loại ảnh rác thải

Hàm phân loại ảnh nhận đầu vào là *buffer* ảnh, chuyển đổi sang định dạng *base64*, sau đó gọi *API Gemini* với *prompt* hướng dẫn phân loại thành 8 loại rác. Kết quả

trả về dạng *JSON* bao gồm loại rác, độ tin cậy, mô tả và hướng dẫn tái chế. Mã nguồn 4.4 minh họa logic này.

Mã nguồn 4.4. Hàm phân loại ảnh rác thải

```
// src/controllers/classifyController.js
const crypto = require('crypto');
const { model } = require('../config/gemini');

// Cache để tránh ọgi API trùng ặlp
const cache = new Map();

async function classifyWaste(imageBuffer, mimeType) {
  // ạTo MD5 hash ủca ảnh để cache ếkt ảqu
  const hash = crypto.createHash('md5')
    .update(imageBuffer).digest('hex');

  if (cache.has(hash)) {
    return cache.get(hash);
  }

  // ểChuyn ảnh sang base64
  const base64Image = imageBuffer.toString('base64');

  // Prompt ướhng ẫdn Gemini phân ạloi 8 ạloi rác
  const prompt = `Phân tích hình ảnh và xác định ạloi rác ảthi.ả
Tr ềv JSON ớvi định ạdng:
{
  "waste_type": "ộmt trong [recyclable, organic, hazardous,
                electronic, glass, metal, paper, plastic]",
  "confidence": ố s ừt 0-1,
  "description": "mô ả t ắngn ọgn ềv ậvt ẩphm",
  "recycling_guide": "ướhng ẫdn tái ếch ặhoc ử x lý"
}`;

  const result = await model.generateContent([
    prompt,
    {
      inlineData: {
        mimeType: mimeType,
        data: base64Image
      }
    }
  ]);

  const response = await result.response;
  const text = response.text();

  // Parse JSON ừt response
```

```
  const jsonMatch = text.match(/\{[\s\S]*\}/);
  const parsed = JSON.parse(jsonMatch[0]);

  // uLu vào cache
  cache.set(hash, parsed);

  return parsed;
}

module.exports = { classifyWaste };
```

4.2.3.3 Chatbot tư vấn tái chế

Chức năng *chatbot* sử dụng *Gemini API* với khả năng duy trì lịch sử hội thoại. Mỗi phiên *chat* được khởi tạo với `startChat()` và gửi tin nhắn qua `sendMessage()`. Mã nguồn 4.5 minh họa cách triển khai.

Mã nguồn 4.5. Chatbot tư vấn tái chế với Gemini

```
// src/controllers/chatController.js
const { model } = require('../config/gemini');

// uLu ữtr các phiên chat theo userId
const chatSessions = new Map();

async function handleChatMessage(userId, message) {
  // ởKhi ạto ặhoc ấly phiên chat ủca user
  if (!chatSessions.has(userId)) {
    const chat = model.startChat({
      history: [],
      generationConfig: {
        maxOutputTokens: 500,
      },
    });
    chatSessions.set(userId, chat);
  }

  const chat = chatSessions.get(userId);

  // ửGi tin ắnhn và ậnhn ảphn ồhi
  const result = await chat.sendMessage(
    `ảTr ời câu ỏhi ề v phân ạloi và tái ếch rác: ${message}`
  );

  const response = await result.response;
  return response.text();
}

```

```
30 module.exports = { handleChatMessage };
```

4.2.4 Xác thực với JWT và bcrypt

Hệ thống sử dụng *JWT* (*JSON Web Token*) để xác thực người dùng và `bcrypt` để mã hóa mật khẩu với 10 *rounds salt*.

4.2.4.1 Đăng ký người dùng

Quy trình đăng ký bao gồm: kiểm tra tính hợp lệ đầu vào, mã hóa mật khẩu bằng `bcrypt`, lưu vào cơ sở dữ liệu, và trả về *JWT token*. Mã nguồn 4.6 mô tả chi tiết.

Mã nguồn 4.6. Đăng ký người dùng với bcrypt và JWT

```
1  // src/controllers/authController.js
2  const bcrypt = require('bcrypt');
3  const jwt = require('jsonwebtoken');
4  const db = require('../config/database');
5
6  async function register(req, res) {
7    const { username, email, password } = req.body;
8
9    // Validate đầu vào
10   if (!username || !email || !password) {
11     return res.status(400).json({
12       error: 'éThiu thông tin ắbt ộbuc'
13     });
14   }
15
16   // ểKim tra email đã ồtn ạti
17   const [existing] = await db.query(
18     'SELECT id FROM users WHERE email = ?',
19     [email]
20   );
21
22   if (existing.length > 0) {
23     return res.status(409).json({
24       error: 'Email đã được ửs ụdng'
25     });
26   }
27
28   // Hash ậmt ẩkhu ởvi 10 rounds
29   const hashedPassword = await bcrypt.hash(password, 10);
30
31   // uLu user vào database
32   const [result] = await db.query(
33     'INSERT INTO users (username, email, password_hash) VALUES (?, ?, ?)',
```

```
    [username, email, hashedPassword]
  );

  // ạTo JWT token có ệhiu ựlc 24 ờgi
  const token = jwt.sign(
    { userId: result.insertId, email },
    process.env.JWT_SECRET,
    { expiresIn: '24h' }
  );

  res.status(201).json({ token, userId: result.insertId });
}

module.exports = { register };
```

4.2.4.2 Đăng nhập

Quá trình đăng nhập tìm kiếm người dùng theo *email*, so sánh mật khẩu đã mã hóa bằng `bcrypt.compare()`, và ký *JWT* có thời hạn 24 giờ. Mã nguồn 4.7 minh họa logic này.

Mã nguồn 4.7. Đăng nhập với xác thực bcrypt

```
// src/controllers/authController.js (ếtip)
async function login(req, res) {
  const { email, password } = req.body;

  // Tìm user theo email
  const [users] = await db.query(
    'SELECT id, username, email, password_hash FROM users WHERE email = ?',
    [email]
  );

  if (users.length === 0) {
    return res.status(401).json({
      error: 'Email ặhoc ậmt ẩkhu không đúng'
    });
  }

  const user = users[0];

  // So sánh password ớvi hash trong database
  const isValidPassword = await bcrypt.compare(
    password,
    user.password_hash
  );

  if (!isValidPassword) {
```

```
    return res.status(401).json({
      error: 'Email ặhoc ậmt ẩkhu không đúng'
    });
  }

  // ạTo JWT token
  const token = jwt.sign(
    { userId: user.id, email: user.email },
    process.env.JWT_SECRET,
    { expiresIn: '24h' }
  );

  res.json({
    token,
    userId: user.id,
    username: user.username
  });
}

module.exports = { register, login };
```

4.2.4.3 Middleware xác thực JWT

Middleware xác thực được áp dụng cho các *route* bảo vệ, kiểm tra và giải mã *JWT* từ *header* `Authorization`. Mã nguồn 4.8 mô tả cách triển khai.

Mã nguồn 4.8. Middleware xác thực JWT

```
// src/middleware/auth.js
const jwt = require('jsonwebtoken');

function authenticateToken(req, res, next) {
  // ấLy token ừt Authorization header
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];

  if (!token) {
    return res.status(401).json({
      error: 'Không tìm ấthy token xác ựthc'
    });
  }

  // Verify token
  jwt.verify(token, process.env.JWT_SECRET, (err, user) => {
    if (err) {
      return res.status(403).json({
        error: 'Token không ợhp ệl ặhoc đã ết ạhn'
      });
```

```
    }

    // uLu thông tin user vào request để ửs ụdng sau
    req.user = user;
    next();
  });
}

module.exports = { authenticateToken };
```

4.2.5 Upload ảnh với multer

Hệ thống sử dụng `multer` để xử lý tải lên ảnh với cấu hình *memory storage* (không lưu file vào đĩa), giới hạn kích thước tối đa 5 MB và chỉ chấp nhận các định dạng ảnh phổ biến. Mã nguồn 4.9 minh họa cấu hình chi tiết.

Mã nguồn 4.9. Cấu hình multer cho upload ảnh

```
// src/middleware/upload.js
const multer = require('multer');

// ửS ụdng memoryStorage để không ulu file vào disk
const storage = multer.memoryStorage();

// Hàm filter ích ấchp ậnhn file ảnh
const fileFilter = (req, file, cb) => {
  const allowedMimes = [
    'image/jpeg',
    'image/png',
    'image/webp'
  ];

  if (allowedMimes.includes(file.mimetype)) {
    cb(null, true);
  } else {
    cb(new Error('íCh ấchp ậnhn file ảnh định ạdng JPEG, PNG ặhoc WebP'),
     false);
  }
};

// ấCu hình multer
const upload = multer({
  storage: storage,
  fileFilter: fileFilter,
  limits: {
    fileSize: 5 * 1024 * 1024 // ỏGii ạhn 5MB
  }
});
```

```
30
31 // Middleware ửx lý ỗli upload
32 const handleUploadError = (err, req, res, next) => {
33   if (err instanceof multer.MulterError) {
34     if (err.code === 'LIMIT_FILE_SIZE') {
35       return res.status(400).json({
36         error: 'File ảnh ượvt quá kích ướthc cho phép (5MB)'
37       });
38     }
39   }
40
41   if (err) {
42     return res.status(400).json({ error: err.message });
43   }
44
45   next();
46 };
47
48 module.exports = { upload, handleUploadError };
```

4.2.6 Rate limiting

Để bảo vệ hệ thống khỏi tấn công *DDoS* và tuân thủ giới hạn của *Gemini API*, hệ thống áp dụng *rate limiting* với hai mức độ khác nhau. Mã nguồn 4.10 mô tả cấu hình.

Mã nguồn 4.10. Cấu hình rate limiting

```
1  // src/middleware/rateLimiter.js
2  const rateLimit = require('express-rate-limit');
3
4  // Rate limiter chung cho ấtt ảc API
5  const generalLimiter = rateLimit({
6    windowMs: 15 * 60 * 1000, // 15 phút
7    max: 100, // ốTi đa 100 requests ỗmi 15 phút
8    message: 'Quá ềnhiu yêu ầcu ừt IP này, vui lòng ửth ại sau',
9    standardHeaders: true,
10   legacyHeaders: false,
11 });
12
13 // Rate limiter nghiêm ặngt cho classify API
14 // (do ốgii ạhn ủca Gemini API free tier)
15 const classifyLimiter = rateLimit({
16   windowMs: 1 * 60 * 1000, // 1 phút
17   max: 10, // ốTi đa 10 requests ỗmi phút
18   message: 'Vui lòng ờch ướtrc khi phân ạloi ảnh ếtip theo',
19   standardHeaders: true,
20   legacyHeaders: false,
```

```
});

module.exports = { generalLimiter, classifyLimiter };
```

4.2.7 Xử lý lỗi và validation

Hệ thống triển khai *middleware* xử lý lỗi toàn cục để đảm bảo tất cả lỗi được xử lý nhất quán và trả về thông báo rõ ràng cho người dùng. Mã nguồn 4.11 minh họa cách triển khai.

Mã nguồn 4.11. Middleware xử lý lỗi toàn cục

```
// src/middleware/errorHandler.js
function errorHandler(err, req, res, next) {
  console.error('Error:', err.stack);

  // ửX lý các ạloi ỗli ục ểth
  if (err.name === 'ValidationError') {
    return res.status(400).json({
      error: 'ữD ệliu không ợhp ệl',
      details: err.message
    });
  }

  if (err.name === 'UnauthorizedError') {
    return res.status(401).json({
      error: 'Không có ềquyn truy ậcp'
    });
  }

  // ỗLi ặmc định
  res.status(err.status || 500).json({
    error: err.message || 'ỗLi server ộni ộb',
    ...(process.env.NODE_ENV === 'development' && {
      stack: err.stack
    })
  });
}

// Hàm validate input ủth công
function validateClassifyInput(req, res, next) {
  if (!req.file) {
    return res.status(400).json({
      error: 'Vui lòng ảti lên file ảnh'
    });
  }
  next();
}
```

```
37
38 module.exports = { errorHandler, validateClassifyInput };
```

4.3 Cài đặt frontend

4.3.1 Cấu trúc thư mục Vue.js

Frontend được xây dựng bằng *Vue.js 3* với *Composition API*, tổ chức theo mô hình *component-based*. Mã nguồn 4.12 mô tả cấu trúc thư mục chi tiết.

Mã nguồn 4.12. Cấu trúc thư mục frontend Vue.js

```
frontend/
|-- src/
|   |-- components/
|   |   |-- ClassifyUpload.vue    # Upload và preview ảnh
|   |   |-- WasteResult.vue       # ểHin ịth ếkt ảqu
|   |   |-- ChatInterface.vue     # Giao ệdin chat
|   |   `-- NavBar.vue
|   |-- views/
|   |   |-- HomeView.vue
|   |   |-- ClassifyView.vue
|   |   |-- ChatView.vue
|   |   |-- HistoryView.vue
|   |   `-- AuthView.vue
|   |-- composables/
|   |   |-- useAuth.js            # Auth state management
|   |   `-- useClassify.js
|   |-- router/index.js
|   |-- api/axios.js              # Axios instance ới vi interceptors
|   `-- main.js
|-- vite.config.js
`-- tailwind.config.js
```

4.3.2 Routing với Vue Router

Vue Router được cấu hình với *navigation guards* để bảo vệ các *route* yêu cầu xác thực. Mã nguồn 4.13 minh họa cấu hình *routes* và *guards*.

Mã nguồn 4.13. Cấu hình Vue Router với navigation guards

```
// src/router/index.js
import { createRouter, createWebHistory } from 'vue-router';
import HomeView from '../views/HomeView.vue';
import ClassifyView from '../views/ClassifyView.vue';
import ChatView from '../views/ChatView.vue';
```

```
import HistoryView from '../views/HistoryView.vue';
import AuthView from '../views/AuthView.vue';

const routes = [
  { path: '/', name: 'Home', component: HomeView },
  {
    path: '/classify',
    name: 'Classify',
    component: ClassifyView,
    meta: { requiresAuth: true } // Route được ảbo ệv
  },
  {
    path: '/chat',
    name: 'Chat',
    component: ChatView,
    meta: { requiresAuth: true }
  },
  {
    path: '/history',
    name: 'History',
    component: HistoryView,
    meta: { requiresAuth: true }
  },
  { path: '/auth', name: 'Auth', component: AuthView }
];

const router = createRouter({
  history: createWebHistory(),
  routes
});

// Navigation guard ểkim tra authentication
router.beforeEach((to, from, next) => {
  const token = localStorage.getItem('token');

  if (to.meta.requiresAuth && !token) {
    // ểChuyn ướhng ềv trang đăng ậnhp ếnu ưcha xác ựthc
    next({ name: 'Auth', query: { redirect: to.fullPath } });
  } else {
    next();
  }
});

export default router;
```

4.3.3 Tích hợp Axios

Axios được cấu hình với *interceptors* để tự động thêm *token* xác thực vào mỗi yêu cầu và xử lý lỗi 401 (chuyển hướng về trang đăng nhập). Mã nguồn 4.14 mô tả cấu hình chi tiết.

Mã nguồn 4.14. Cấu hình Axios với interceptors

```
// src/api/axios.js
import axios from 'axios';
import router from '../router';

// ạTo axios instance ớvi baseURL ừt environment
const apiClient = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL || 'http://localhost:3000/api',
  timeout: 30000,
  headers: {
    'Content-Type': 'application/json'
  }
});

// Request interceptor: thêm Authorization header
apiClient.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem('token');
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => {
    return Promise.reject(error);
  }
);

// Response interceptor: ửx lý ỗli 401
apiClient.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response && error.response.status === 401) {
      // Xóa token và ểchuyn ềv trang đăng ậnhp
      localStorage.removeItem('token');
      localStorage.removeItem('userId');
      router.push({ name: 'Auth' });
    }
    return Promise.reject(error);
  }
);

export default apiClient;
```

4.3.4 Upload ảnh và preview

Component `ClassifyUpload.vue` cho phép người dùng tải lên ảnh thông qua `input[type=file]` hoặc kéo thả (*drag and drop*). Ảnh được *preview* bằng `FileReader` trước khi gửi lên *server* dưới dạng `FormData`. Mã nguồn 4.15 minh họa logic xử lý.

Mã nguồn 4.15. Logic upload và preview ảnh

```
// src/components/ClassifyUpload.vue (script section)
import { ref } from 'vue';
import apiClient from '../api/axios';

const selectedFile = ref(null);
const previewUrl = ref(null);
const isUploading = ref(false);

// ửX lý khi ọchn file
function handleFileSelect(event) {
  const file = event.target.files[0];
  if (!file) return;

  // ểKim tra ạloi file
  if (!file.type.startsWith('image/')) {
    alert('Vui lòng ọchn file ảnh');
    return;
  }

  // ểKim tra kích ướthc (max 5MB)
  if (file.size > 5 * 1024 * 1024) {
    alert('Ảnh ượvt quá kích ướthc cho phép (5MB)');
    return;
  }

  selectedFile.value = file;

  // ạTo preview URL ằbng FileReader
  const reader = new FileReader();
  reader.onload = (e) => {
    previewUrl.value = e.target.result;
  };
  reader.readAsDataURL(file);
}

// Upload ảnh lên server để phân ạloi
async function uploadAndClassify() {
  if (!selectedFile.value) return;

  isUploading.value = true;

```

```
  // ạTo FormData để ửgi file
  const formData = new FormData();
  formData.append('image', selectedFile.value);

  try {
    const response = await apiClient.post('/classify', formData, {
      headers: {
        'Content-Type': 'multipart/form-data'
      }
    });

    // Emit ếkt ảqu cho component cha
    emit('classifyResult', response.data);
  } catch (error) {
    console.error('ỗLi phân ạloi:', error);
    alert(error.response?.data?.error || 'Có ỗli ảxy ra');
  } finally {
    isUploading.value = false;
  }
}
```

4.3.5 Hiển thị kết quả phân loại

Component `WasteResult.vue` nhận kết quả từ *API* và hiển thị loại rác với màu tương ứng, thanh *confidence*, mô tả và hướng dẫn tái chế. Màu sắc cho 8 loại rác được định nghĩa trong file `constants.js`. Mã nguồn 4.16 liệt kê các màu được sử dụng.

Mã nguồn 4.16. Định nghĩa màu sắc cho 8 loại rác

```
// src/constants.js
export const WASTE_TYPE_COLORS = {
  recyclable: { bg: 'bg-blue-500', text: 'text-blue-700' },
  organic: { bg: 'bg-green-500', text: 'text-green-700' },
  hazardous: { bg: 'bg-red-500', text: 'text-red-700' },
  electronic: { bg: 'bg-purple-500', text: 'text-purple-700' },
  glass: { bg: 'bg-cyan-500', text: 'text-cyan-700' },
  metal: { bg: 'bg-gray-500', text: 'text-gray-700' },
  paper: { bg: 'bg-yellow-500', text: 'text-yellow-700' },
  plastic: { bg: 'bg-orange-500', text: 'text-orange-700' }
};

export const WASTE_TYPE_NAMES = {
  recyclable: 'Rác tái ếch',
  organic: 'Rác ữhu ơc',
  hazardous: 'Rác nguy ạhi',
  electronic: 'Rác điện ửt',
  glass: 'ủThy tinh',
  metal: 'Kim ạloi',
```

```
  paper: 'ấGiy',
  plastic: 'ựNha'
};
```

4.3.6 Cấu hình PWA

Ứng dụng được cấu hình thành *Progressive Web App* (*PWA*) sử dụng `vite-plugin-pwa` với chiến lược *generateSW*. File `manifest.json` định nghĩa tên ứng dụng, biểu tượng và màu chủ đề. Mã nguồn 4.17 mô tả cấu hình *PWA*.

Mã nguồn 4.17. Cấu hình PWA với Vite

```
// vite.config.js
import { defineConfig } from 'vite';
import vue from '@vitejs/plugin-vue';
import { VitePWA } from 'vite-plugin-pwa';

export default defineConfig({
  plugins: [
    vue(),
    VitePWA({
      registerType: 'autoUpdate',
      includeAssets: ['favicon.ico', 'robots.txt', 'apple-touch-icon.png'],
      manifest: {
        name: 'ệH ốthng phân ạloi rác ảthi thông minh',
        short_name: 'WasteClassify',
        description: 'Ứng ụdng phân ạloi rác ảthi ằbng AI',
        theme_color: '#10b981',
        background_color: '#ffffff',
        display: 'standalone',
        icons: [
          {
            src: '/icon-192x192.png',
            sizes: '192x192',
            type: 'image/png'
          },
          {
            src: '/icon-512x512.png',
            sizes: '512x512',
            type: 'image/png'
          }
        ]
      },
      workbox: {
        // NetworkFirst cho API requests
        runtimeCaching: [
          {
            urlPattern: /^https:\/\/.*\/api\/.*/,
```

```
            handler: 'NetworkFirst',
            options: {
              cacheName: 'api-cache',
              expiration: {
                maxEntries: 50,
                maxAgeSeconds: 300 // 5 phút
              }
            }
          },
          {
            // CacheFirst cho static assets
            urlPattern: /\.(js|css|png|jpg|jpeg|svg|webp)$/,
            handler: 'CacheFirst',
            options: {
              cacheName: 'assets-cache',
              expiration: {
                maxEntries: 100,
                maxAgeSeconds: 7 * 24 * 60 * 60 // 7 ngày
              }
            }
          }
        ]
      }
    })
  ]
});
```

4.4 Cơ sở dữ liệu

Hệ thống sử dụng *MySQL* làm cơ sở dữ liệu quan hệ. *Schema* bao gồm bốn bảng chính: `users`, `waste_types`, `classification_history` và `chat_history`. Mã nguồn 4.18 hiển thị *SQL* tạo *schema* đầy đủ.

Mã nguồn 4.18. Schema cơ sở dữ liệu MySQL

```
-- ạTo database
CREATE DATABASE IF NOT EXISTS waste_classification
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

USE waste_classification;

-- ảBng ườngi dùng
CREATE TABLE users (
  id INT PRIMARY KEY AUTO_INCREMENT,
  username VARCHAR(50) NOT NULL UNIQUE,
  email VARCHAR(100) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
```

```
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_email (email)
) ENGINE=InnoDB;

-- ảBng ạloi rác ảthi
CREATE TABLE waste_types (
  id INT PRIMARY KEY AUTO_INCREMENT,
  type_code VARCHAR(20) NOT NULL UNIQUE,
  name_vi VARCHAR(100) NOT NULL,
  name_en VARCHAR(100) NOT NULL,
  description TEXT,
  recycling_guide TEXT,
  color_code VARCHAR(7),
  icon_path VARCHAR(255)
) ENGINE=InnoDB;

-- ảBng ịlch ửs phân ạloi
CREATE TABLE classification_history (
  id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT NOT NULL,
  waste_type_id INT,
  image_hash VARCHAR(32) NOT NULL,
  confidence DECIMAL(5,4),
  result_json JSON,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY (waste_type_id) REFERENCES waste_types(id) ON DELETE SET NULL,
  INDEX idx_user_date (user_id, created_at),
  INDEX idx_image_hash (image_hash)
) ENGINE=InnoDB;

-- ảBng ịlch ửs chat
CREATE TABLE chat_history (
  id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT NOT NULL,
  message TEXT NOT NULL,
  response TEXT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  INDEX idx_user_date (user_id, created_at)
) ENGINE=InnoDB;

-- Thêm ữd ệliu ẫmu cho 8 ạloi rác
INSERT INTO waste_types (type_code, name_vi, name_en, description,
  recycling_guide, color_code) VALUES
('recyclable', 'Rác tái ếch', 'Recyclable',
  'Rác có ểth tái ếch thành ảsn ẩphm ơmi',
  'ửRa ạsch, tách nhãn, ỏb vào thùng xanh ươdng', '#3b82f6'),
```

```
('organic', 'Rác ữhu ơc', 'Organic',
  'Rác có ồngun ốgc sinh ọhc có ểth phân ủhy',
  'ỏB vào thùng xanh lá, có ểth làm phân compost', '#10b981'),
('hazardous', 'Rác nguy ạhi', 'Hazardous',
  'Rác ứcha ấcht độc ạhi, nguy ểhim',
  'Không ứvt chung, giao cho đơn ịv chuyên ửx lý', '#ef4444'),
('electronic', 'Rác đệin ửt', 'Electronic',
  'ếThit ịb đệin ửt ỏhng ặhoc ũc',
  'Mang đến ểđim thu gom đệin ửt ặhoc ửca hàng thu ồhi', '#a855f7'),
('glass', 'ủThy tinh', 'Glass',
  'Chai ọl ủthy tinh',
  'ửRa ạsch, tách ắnp, ỏb vào thùng riêng', '#06b6d4'),
('metal', 'Kim ạloi', 'Metal',
  'Lon, ộhp kim ạloi',
  'ửRa ạsch, ẹdp ẳphng, ỏb vào thùng kim ạloi', '#6b7280'),
('paper', 'ấGiy', 'Paper',
  'ấGiy, bìa carton',
  'ấGp ọgn, ữgi khô, ỏb vào thùng ấgiy', '#eab308'),
('plastic', 'ựNha', 'Plastic',
  'Chai ọl, túi nilon ựnha',
  'ửRa ạsch, phân ạloi theo ốs ký ệhiu, ỏb vào thùng ựnha', '#f97316');
```

4.5 Triển khai hệ thống

4.5.1 Build production

Quá trình *build* cho môi trường *production* bao gồm biên dịch *frontend* thành các file tĩnh và chạy *backend* với *process manager*.

4.5.1.1 Build frontend

Lệnh `npm run build` biên dịch *Vue.js* thành các file *HTML*, *CSS* và *JavaScript* tối ưu trong thư mục `dist/`. Các file này có thể được phục vụ bởi *web server* như *Nginx* hoặc *Apache*.

Mã nguồn 4.19. Build frontend production

```
cd frontend
npm run build

# ếKt ảqu trong dist/
# dist/
# |-- index.html
# |-- assets/
# |   |-- index-[hash].js
# |   `-- index-[hash].css
```

```
# `-- manifest.webmanifest
```

4.5.1.2 Chạy backend

Backend có thể chạy trực tiếp với `node` hoặc sử dụng *PM2* để quản lý tiến trình và tự động khởi động lại khi có lỗi.

Mã nguồn 4.20. Chạy backend với PM2

```
# ạChy ựtrc ếtip
node src/app.js

# ặHoc dùng PM2 cho production
npm install -g pm2
pm2 start src/app.js --name waste-classify-api
pm2 save
pm2 startup
```

4.5.2 Cấu hình environment variables

File `.env` chứa các biến môi trường cần thiết cho hệ thống. Mã nguồn 4.21 hiển thị *template* file `.env`.

Mã nguồn 4.21. Template file .env

```
# Server configuration
PORT=3000
NODE_ENV=production

# Database configuration
DB_HOST=localhost
DB_PORT=3306
DB_USER=waste_user
DB_PASSWORD=secure_password_here
DB_NAME=waste_classification

# Google Gemini AI
GEMINI_API_KEY=your_gemini_api_key_here

# JWT Secret
JWT_SECRET=your_jwt_secret_key_here

# CORS Origin
CORS_ORIGIN=https://your-domain.com
```

4.5.3 Cấu hình CORS

Backend cấu hình *CORS* (*Cross-Origin Resource Sharing*) để cho phép *frontend* gọi *API* từ *domain* khác nhau. Mã nguồn 4.22 mô tả cấu hình chi tiết.

Mã nguồn 4.22. Cấu hình CORS cho Express

```
// src/app.js
const express = require('express');
const cors = require('cors');

const app = express();

// ấCu hình CORS
const corsOptions = {
  origin: function (origin, callback) {
    // Whitelist các domain được phép
    const allowedOrigins = [
      'https://waste-classify.com',
      'https://www.waste-classify.com',
      process.env.NODE_ENV === 'development' && 'http://localhost:5173'
    ].filter(Boolean);

    if (!origin || allowedOrigins.includes(origin)) {
      callback(null, true);
    } else {
      callback(new Error('Not allowed by CORS'));
    }
  },
  methods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'],
  allowedHeaders: ['Content-Type', 'Authorization'],
  credentials: true,
  maxAge: 86400 // 24 ờgi
};

app.use(cors(corsOptions));

// Các middleware và routes khác...
```

Chương này đã trình bày chi tiết quá trình cài đặt hệ thống từ môi trường phát triển, cấu trúc và triển khai *backend* với *Node.js*, *Express*, *MySQL* và *Google Gemini AI*, đến *frontend* với *Vue.js*, xác thực *JWT*, xử lý tải lên ảnh, và triển khai thành *PWA*. Tất cả các thành phần được tích hợp chặt chẽ để tạo nên một hệ thống hoàn chỉnh, sẵn sàng cho giai đoạn kiểm thử và triển khai thực tế.

CHƯƠNG 5

Kết quả và đánh giá

MỤC TIÊU CHƯƠNG

Chương này trình bày kết quả thực tế đạt được của hệ thống phân loại rác thải tự động, bao gồm các chức năng đã hoàn thành, đánh giá độ chính xác phân loại trên từng loại rác, hiệu năng hệ thống, khảo sát trải nghiệm người dùng, so sánh với mục tiêu đề ra, cũng như phân tích ưu điểm, hạn chế và hướng phát triển tiếp theo.

5.1 Kết quả đạt được

5.1.1 Tóm tắt chức năng đã hoàn thành

Hệ thống đã hoàn thành tất cả các chức năng chính đề ra trong Chương 1. Bảng 5.1 liệt kê chi tiết các chức năng và trạng thái triển khai.

Bảng 5.1. Danh sách chức năng và trạng thái hoàn thành.

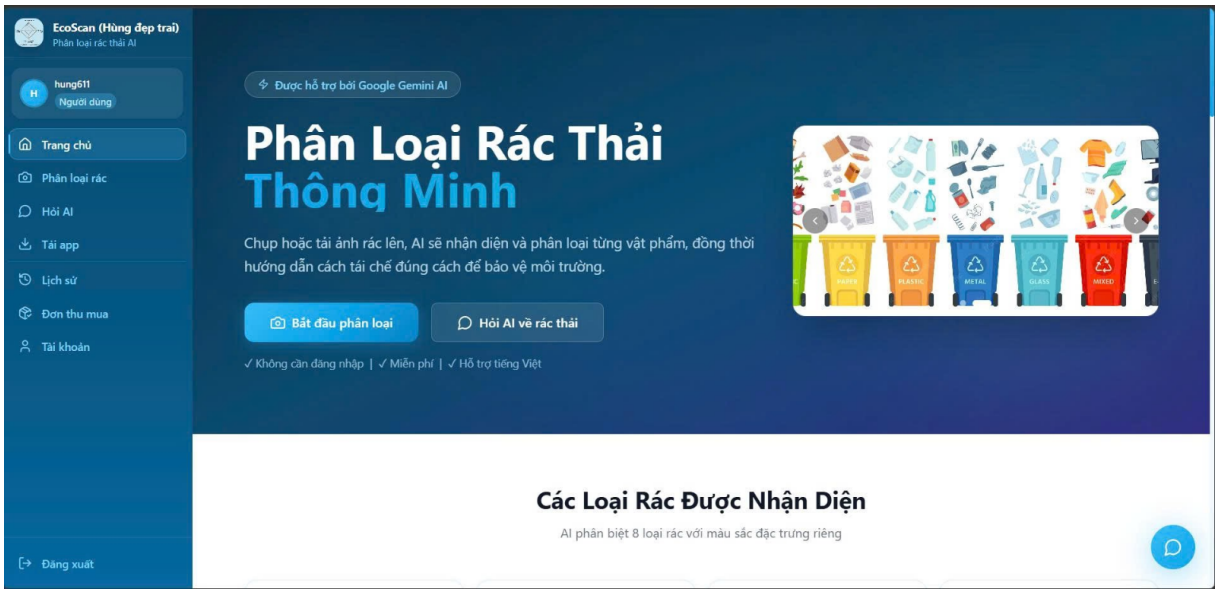
STT	Chức năng	Trạng thái	Ghi chú
1	Đăng ký/đăng nhập JWT	Hoàn thành	Bảo mật token
2	Phân loại ảnh 8 loại với Gemini Vision	Hoàn thành	API tích hợp
3	Hiển thị kết quả + confidence score	Hoàn thành	Độ tin cậy %
4	Hướng dẫn tái chế theo loại rác	Hoàn thành	Chi tiết theo loại
5	Chatbot tư vấn AI	Hoàn thành	Gemini API
6	Lịch sử phân loại	Hoàn thành	Lưu trữ cơ sở dữ liệu
7	Giao diện responsive (mobile/desktop)	Hoàn thành	Responsive CSS
8	Cài đặt ứng dụng di động	Có hướng dẫn	iOS qua Safari, Android APK
9	Cache MD5 tránh gọi API trùng	Hoàn thành	Tối ưu chi phí
10	Quản lý profile người dùng	Hoàn thành	Cập nhật thông tin
11	Chat với nhân viên hỗ trợ	Có giao diện	Kênh hỗ trợ trực tiếp
12	Tạo và theo dõi đơn thu mua	Có giao diện	Theo dõi trạng thái đơn
13	Bảng giá và cài đặt thu mua	Có giao diện	Giá, địa chỉ, hướng dẫn
14	Quản trị hệ thống	Có giao diện	Người dùng, nhân viên, chat, thống kê
15	Quản lý ứng dụng và phản hồi	Có giao diện	Banner, link cài đặt, đánh giá

Các giao diện được cung cấp còn thể hiện kênh chat với nhân viên bên cạnh chatbot AI; trang tài khoản gồm thông tin cá nhân, bảo mật, đánh giá và phản hồi; trang đơn thu mua có tiến trình trạng thái; cùng màn hình quản lý bảng giá và thông tin nhận hàng. Khu vực quản trị có các mục quản lý người dùng, nhân

viên, chat, ảnh tải lên, banner, thống kê AI, đánh giá, đơn thu mua, giá thu mua, cài đặt ứng dụng và phản hồi. Màn hình tải ứng dụng hướng dẫn cài trên iOS qua Safari và cung cấp liên kết APK Android. Các nội dung này được ghi nhận theo giao diện; trạng thái xử lý phía máy chủ sẽ được xác nhận khi có mã nguồn.

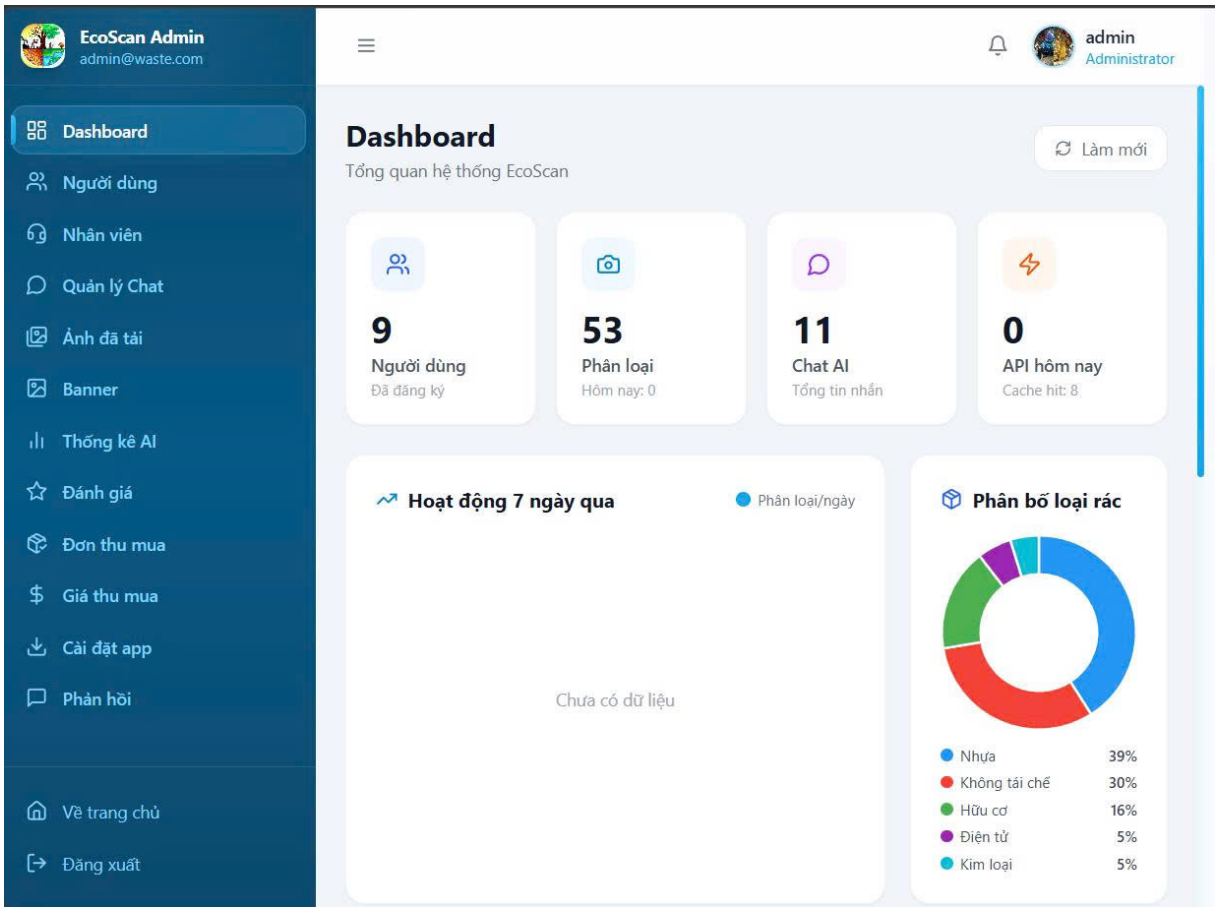
5.1.2 Minh họa giao diện ứng dụng

Các ảnh dưới đây minh họa ba khu vực tiêu biểu của EcoScan: giao diện người dùng, trang tổng quan quản trị và hộp thư hỗ trợ. Mỗi hình được nhận xét theo thông tin và chức năng quan sát được trên giao diện.



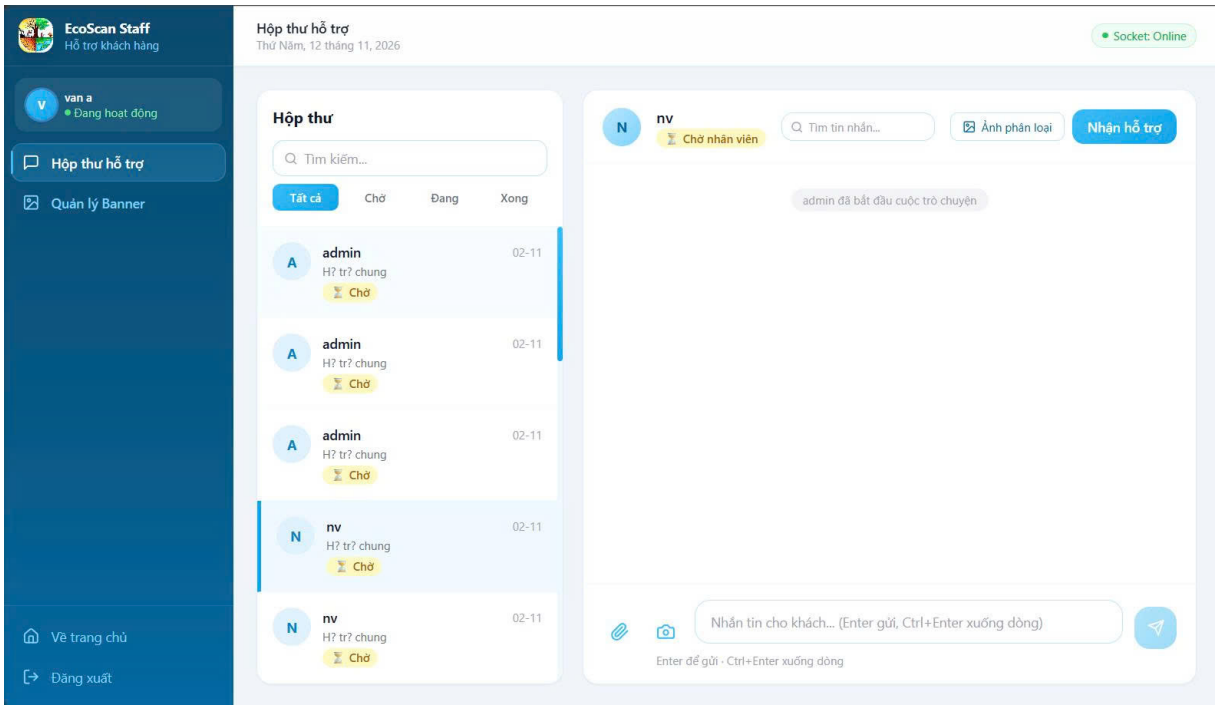
Hình 5.1. Trang chủ EcoScan với lối tắt đến chức năng phân loại rác và hỏi AI.

Trang chủ giới thiệu trực tiếp chức năng cốt lõi, với lối tắt để bắt đầu phân loại hoặc hỏi AI; thanh điều hướng giúp chuyển nhanh sang lịch sử, đơn thu mua và tài khoản. Cách bố trí này giúp người dùng mới nhận ra việc cần làm ngay, đồng thời vẫn truy cập được các chức năng phụ trợ.



Hình 5.2. Bảng điều khiển quản trị với thống kê và biểu đồ hoạt động của hệ thống.

Dashboard tập trung các chỉ số người dùng, lượt phân loại, tin nhắn AI và lượt gọi API, kèm biểu đồ hoạt động theo thời gian và phân bố loại rác. Nhờ đó, người quản trị có thể theo dõi tình hình sử dụng và nhận biết nhanh khi dữ liệu hoạt động thay đổi; các con số trong ảnh là trạng thái tại thời điểm chụp, không phải kết quả đánh giá hiệu năng của báo cáo.

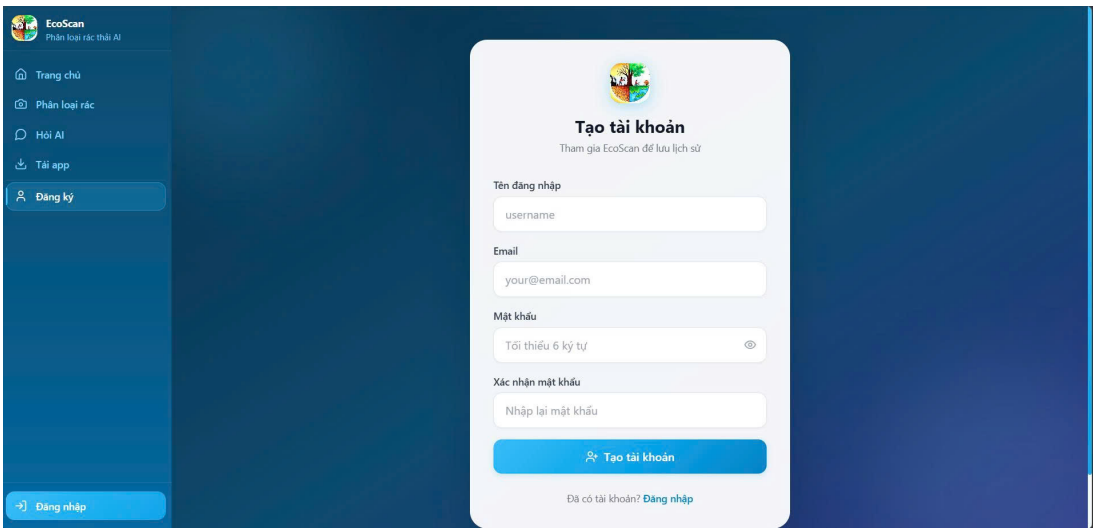


Hình 5.3. Màn hình nhân viên theo dõi và trả lời yêu cầu hỗ trợ của khách hàng.

Hộp thư hỗ trợ hiển thị danh sách hội thoại theo trạng thái và nội dung trao đổi đang được chọn. Nhân viên có thể tìm hội thoại, gửi phản hồi, đính kèm ảnh phân loại và kết thúc phiên hỗ trợ; trạng thái kết nối trực tuyến cũng được hiển thị. Việc tách danh sách khỏi khung hội thoại giúp nhân viên xử lý yêu cầu liên tục mà vẫn nắm được ngữ cảnh trao đổi.

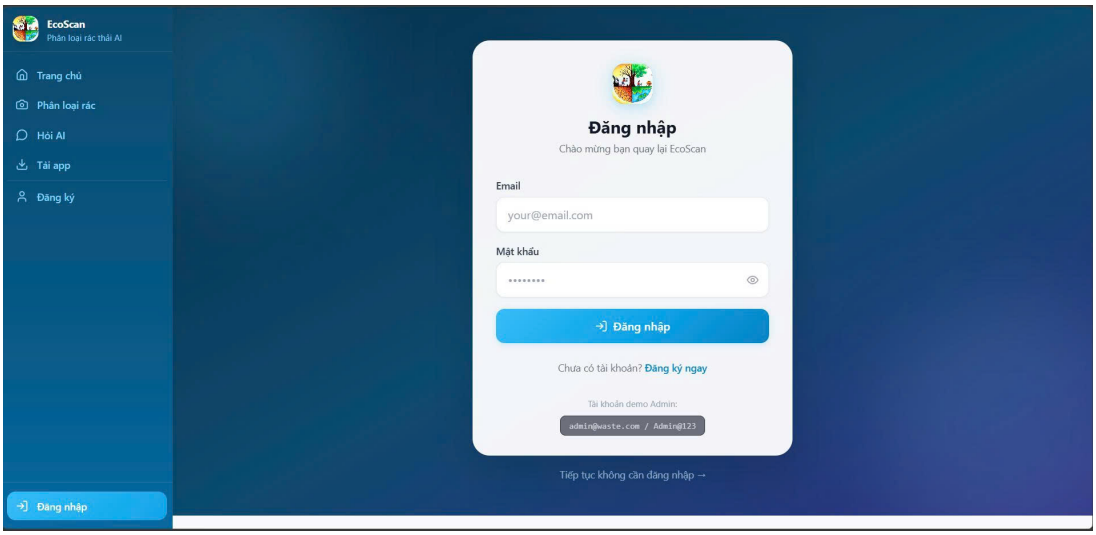
5.1.3 Các giao diện chức năng người dùng và quản trị

Các hình tiếp theo bổ sung những trang chức năng còn lại. Mỗi ảnh được trình bày thành một hình riêng, kèm chú thích và diễn giải ngay bên dưới.



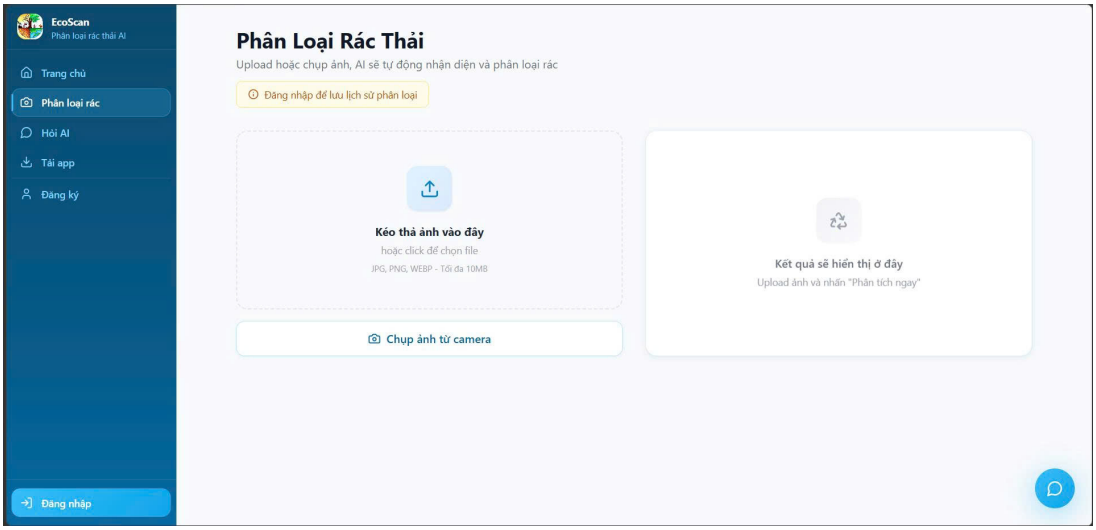
Hình 5.4. Màn hình đăng ký tài khoản EcoScan.

Diễn giải: Biểu mẫu thu thập tên đăng nhập, email và mật khẩu, đồng thời yêu cầu xác nhận mật khẩu. Các trường thông tin được gom trong một luồng để người dùng tạo tài khoản và sử dụng các chức năng cần lưu lịch sử.



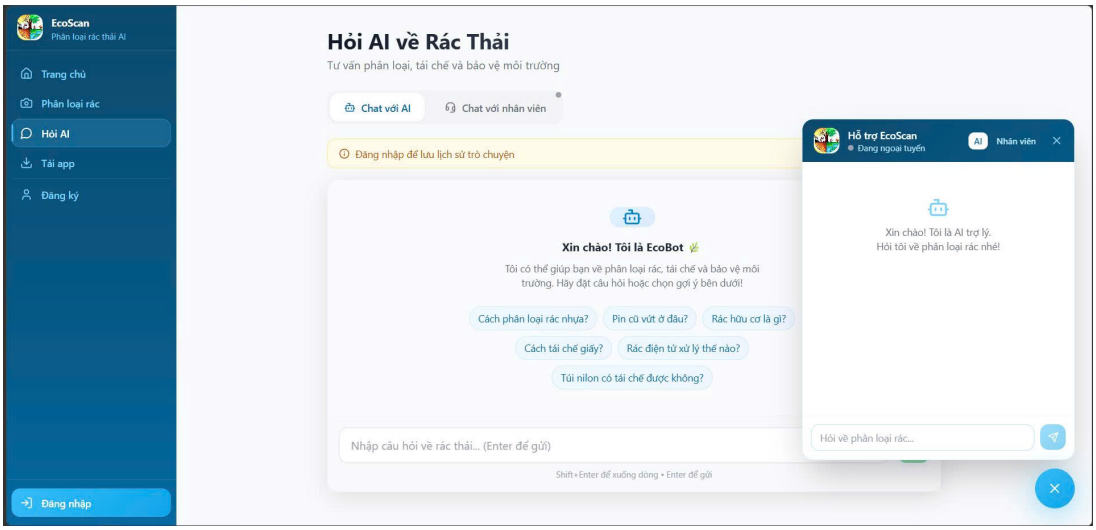
Hình 5.5. Màn hình đăng nhập EcoScan.

Diễn giải: Người dùng nhập email và mật khẩu để truy cập tài khoản; màn hình cũng liên kết sang đăng ký và cho phép tiếp tục khi chưa đăng nhập. Cách phân tách này hỗ trợ cả người dùng mới lẫn người đã có tài khoản.



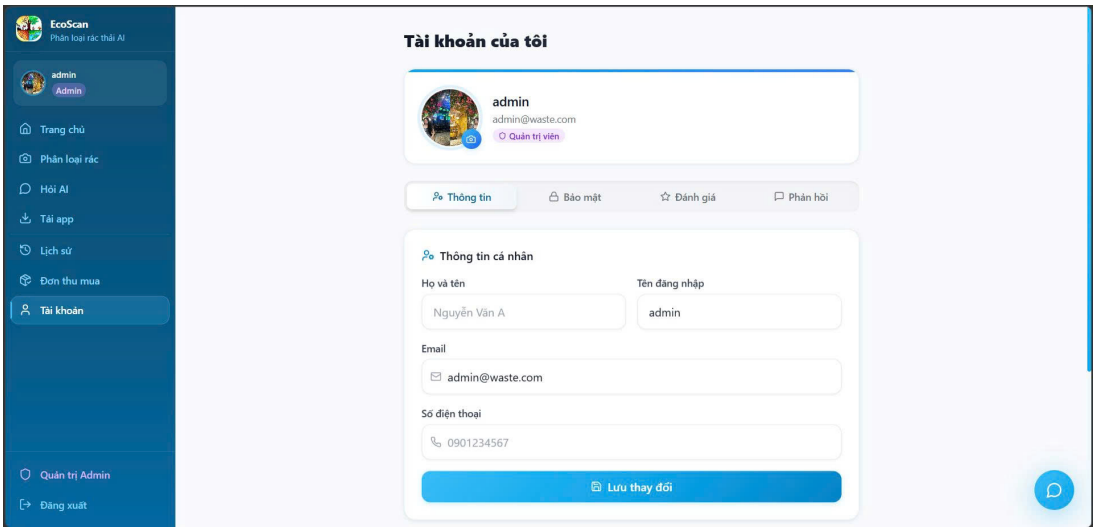
Hình 5.6. Giao diện tải ảnh để phân loại rác.

Diễn giải: Khu vực tải ảnh hỗ trợ kéo thả, chọn tệp và chụp bằng camera; khung bên cạnh dành cho kết quả. Tách vùng nhập ảnh khỏi vùng kết quả giúp người dùng nhận biết rõ đầu vào và đầu ra của chức năng chính.



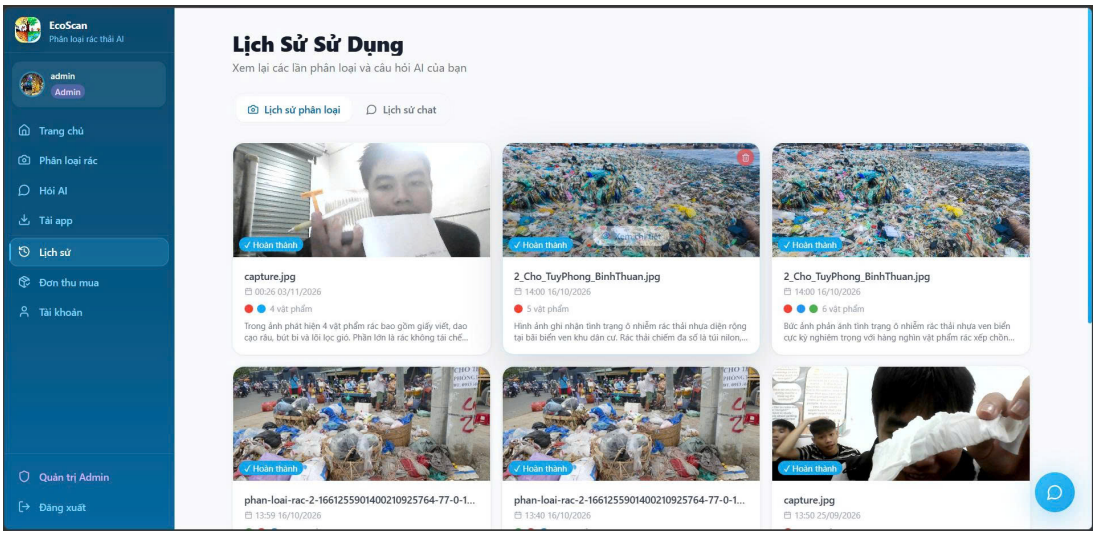
Hình 5.7. Giao diện hỏi đáp AI và hỗ trợ nhân viên.

Diễn giải: Người dùng có thể chọn chat AI hoặc chat với nhân viên; các gợi ý câu hỏi giúp bắt đầu nhanh. Hai kênh được đặt cùng trang để phân biệt tư vấn tự động với hỗ trợ từ nhân viên.



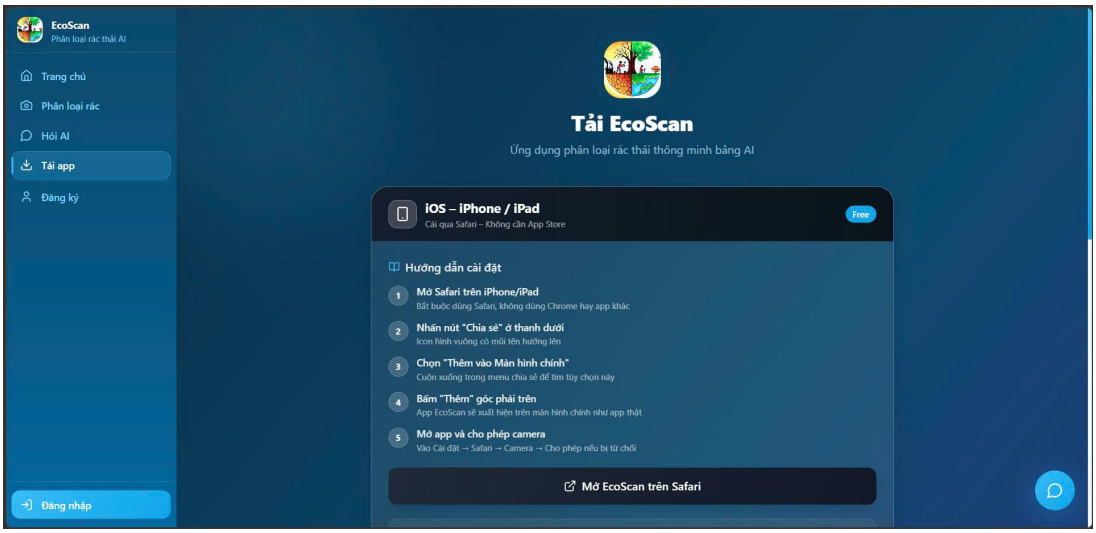
Hình 5.8. Trang thông tin tài khoản.

Diễn giải: Trang hồ sơ hiển thị thông tin cá nhân và các tab bảo mật, đánh giá, phản hồi. Việc tập trung các thiết lập tài khoản tại một nơi giúp người dùng quản lý thông tin và tương tác của mình thuận tiện hơn.



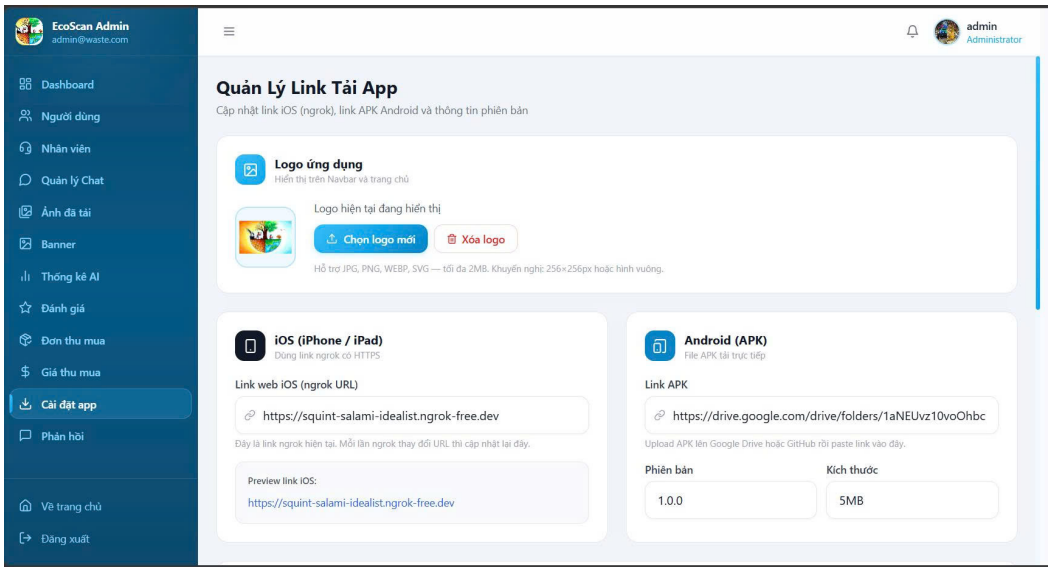
Hình 5.9. Lịch sử phân loại và trò chuyện.

Diễn giải: Danh sách lưu lại các lần phân loại và có tab xem lịch sử chat; mỗi mục thể hiện ảnh, thời điểm và thông tin kết quả. Lịch sử giúp người dùng tra cứu lại hoạt động thay vì phải thực hiện lại thao tác.



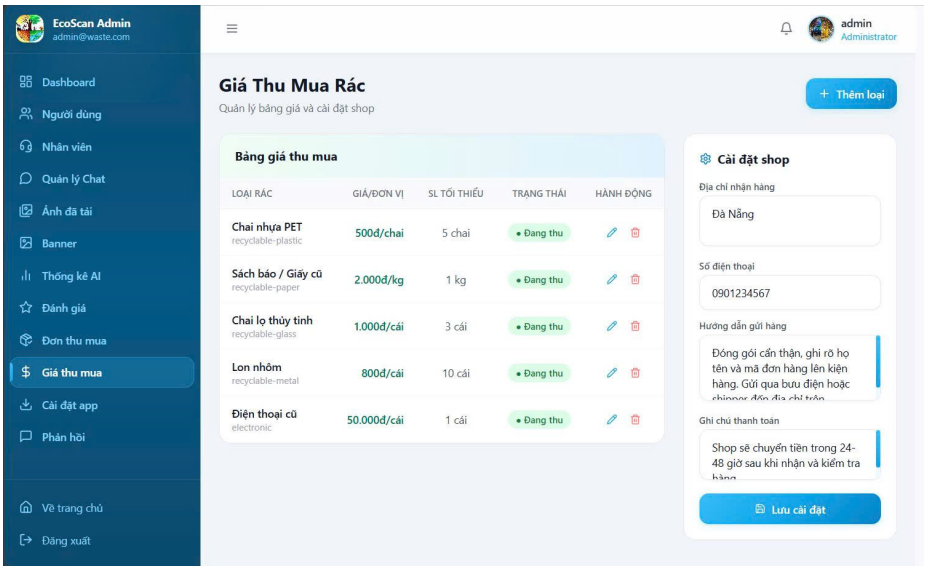
Hình 5.10. Hướng dẫn cài EcoScan trên iOS.

Diễn giải: Màn hình hướng dẫn các bước mở Safari, dùng chức năng chia sẻ và thêm ứng dụng vào màn hình chính. Hướng dẫn theo đúng thao tác của iOS giúp người dùng cài PWA mà không cần tìm ứng dụng trên App Store.



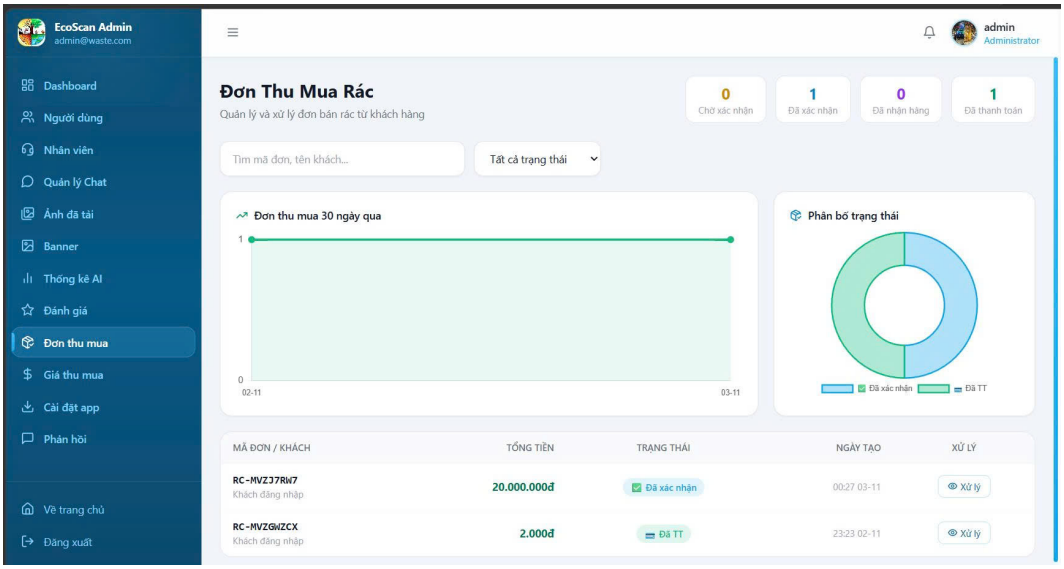
Hình 5.11. Cấu hình logo và liên kết tải ứng dụng.

Diễn giải: Quản trị viên cập nhật logo, liên kết web iOS và tệp APK Android, đồng thời quản lý thông tin phiên bản. Tập trung các cấu hình tải ứng dụng giúp nội dung hướng dẫn và đường dẫn phân phối được cập nhật tại một màn hình.



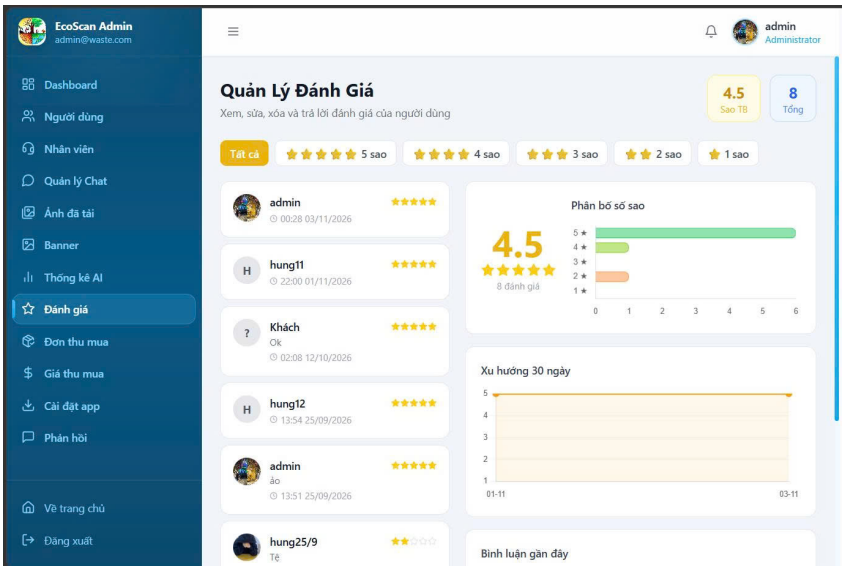
Hình 5.12. Quản lý giá thu mua và thông tin cửa hàng.

Diễn giải: Bảng giá thể hiện loại rác, đơn vị tính, số lượng tối thiểu và trạng thái thu mua; phần cài đặt chứa địa chỉ, số điện thoại và hướng dẫn giao hàng. Thông tin này giúp người dùng biết điều kiện thu mua trước khi tạo đơn.



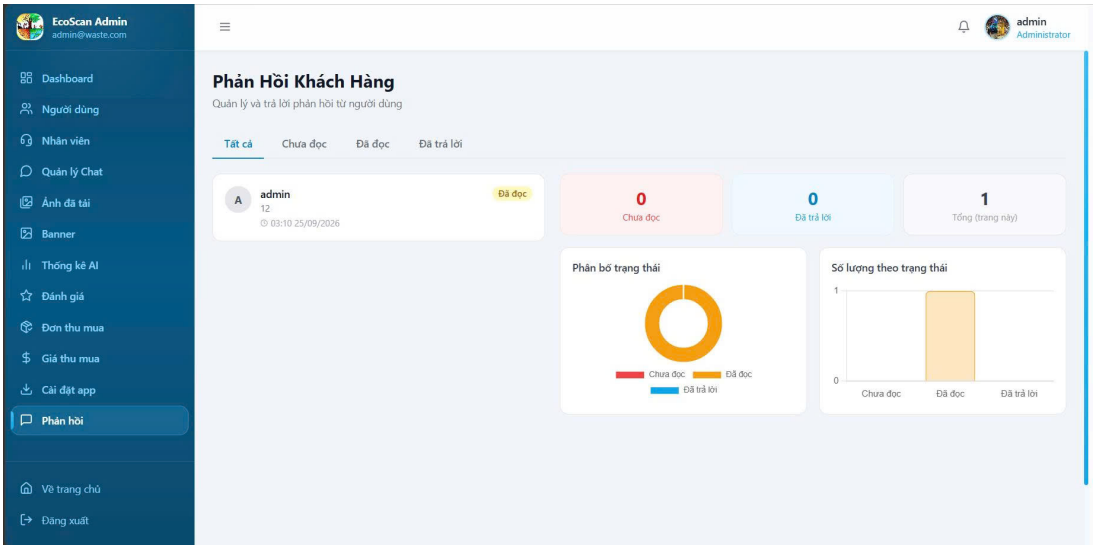
Hình 5.13. Quản lý đơn thu mua rác.

Diễn giải: Trang quản trị tổng hợp số đơn theo trạng thái, biểu đồ và danh sách đơn có tìm kiếm, lọc. Nhân viên có thể theo dõi các bước xác nhận và thanh toán, hỗ trợ xử lý đơn tập trung.



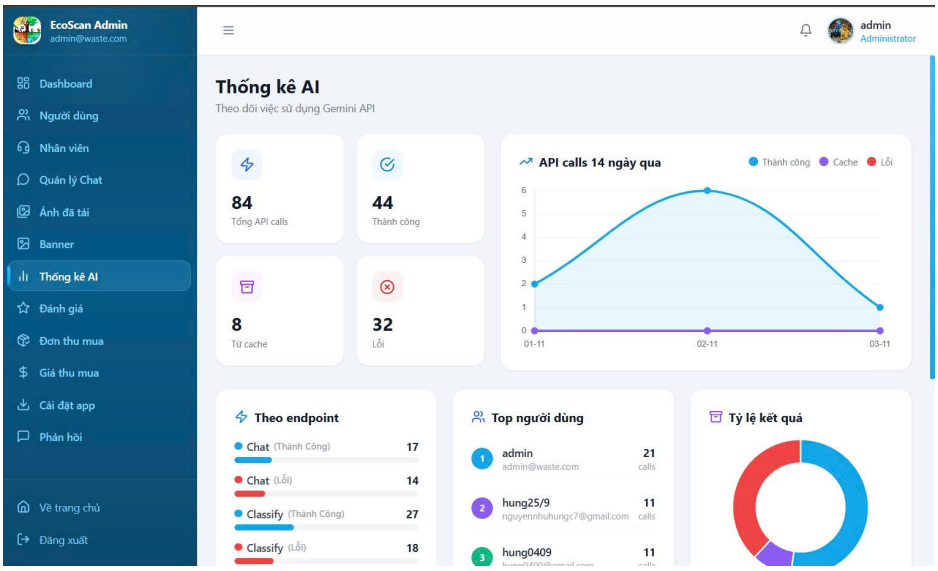
Hình 5.14. Quản lý đánh giá của người dùng.

Diễn giải: Màn hình tổng hợp điểm trung bình, số lượng đánh giá, phân bố số sao và xu hướng theo thời gian. Các bộ lọc theo mức sao giúp quản trị viên xem phản hồi và nhận biết vấn đề trải nghiệm.



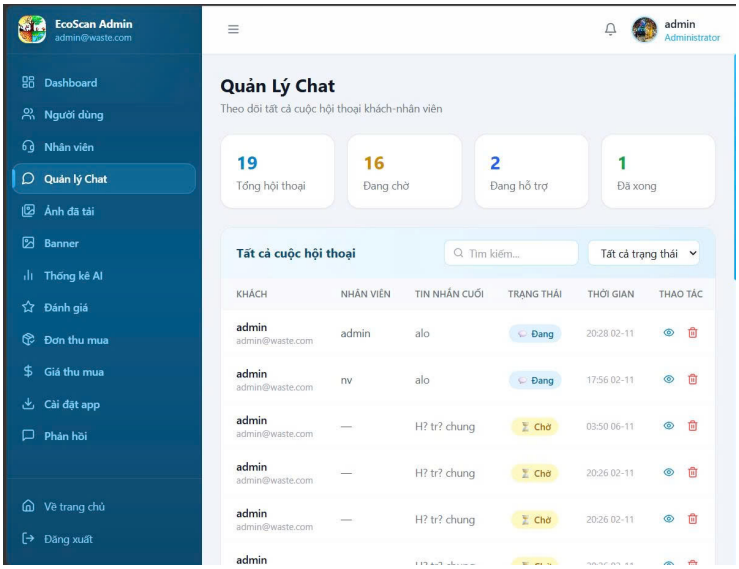
Hình 5.15. Quản lý phản hồi khách hàng.

Diễn giải: Danh sách phản hồi được phân nhóm theo trạng thái đã đọc và đã trả lời, kèm số liệu tổng quan. Cách phân loại này giúp nhân viên ưu tiên phản hồi chưa xử lý và theo dõi tiến độ tiếp nhận.



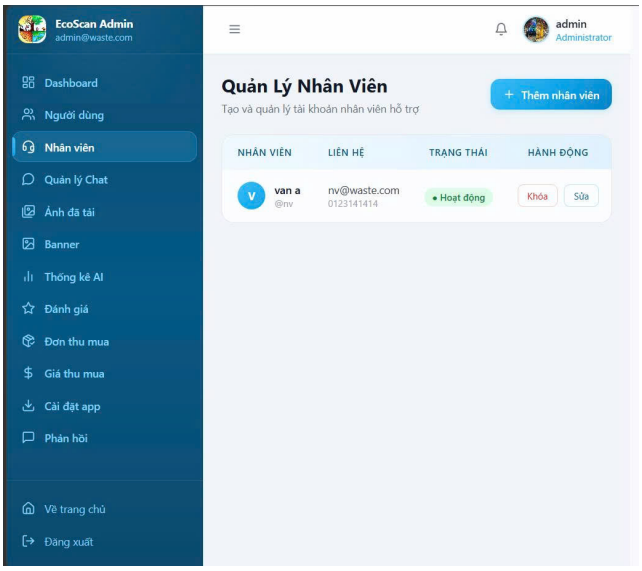
Hình 5.16. Thống kê lượt sử dụng Gemini API.

Diễn giải: Dashboard thể hiện tổng lượt gọi, thành công, cache và lỗi; đồng thời phân tích theo endpoint và người dùng. Các chỉ số hỗ trợ theo dõi hoạt động dịch vụ AI, nhưng số liệu trong ảnh chỉ phản ánh thời điểm chụp.



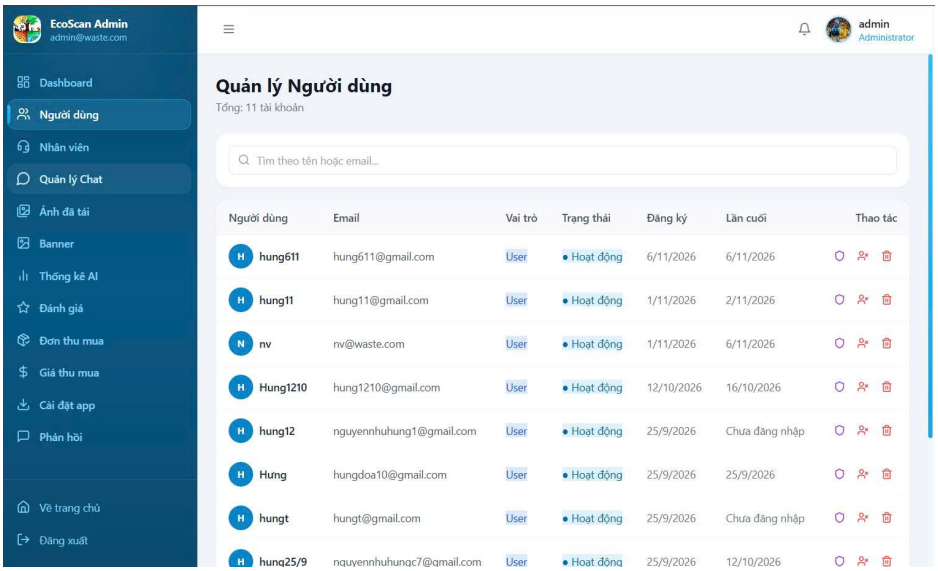
Hình 5.17. Quản lý hội thoại hỗ trợ.

Diễn giải: Quản trị viên có thể tìm kiếm hội thoại khách hàng–nhân viên, lọc theo trạng thái và xem thông tin phiên. Việc theo dõi tập trung giúp kiểm soát các yêu cầu đang chờ, đang hỗ trợ hoặc đã hoàn tất.



Hình 5.18. Quản lý tài khoản nhân viên hỗ trợ.

Diễn giải: Bảng nhân viên hiển thị thông tin liên hệ, trạng thái hoạt động và thao tác khóa hoặc sửa. Màn hình cho phép quản lý quyền truy cập của nhóm phụ trách hỗ trợ khách hàng.



Hình 5.19. Quản lý người dùng hệ thống.

Diễn giải: Danh sách tài khoản có tìm kiếm theo tên hoặc email, kèm vai trò, trạng thái và lần đăng nhập gần nhất. Các thao tác quản trị giúp theo dõi tài khoản và xử lý trường hợp cần hạn chế quyền truy cập.

5.2 Đánh giá độ chính xác phân loại

5.2.1 Phương pháp kiểm thử

Để đánh giá độ chính xác của mô hình phân loại, nhóm đã tiến hành thử nghiệm trên bộ dữ liệu gồm 400 ảnh, với 50 ảnh cho mỗi loại rác trong tổng số 8 loại. Các ảnh được thu thập từ nhiều nguồn khác nhau bao gồm ảnh chụp thực tế, ảnh tải từ internet và ảnh từ các bộ dữ liệu công khai về rác thải. Điều kiện chụp đa dạng về góc độ, ánh sáng và độ phân giải để mô phỏng các tình huống thực tế khi người dùng sử dụng ứng dụng.

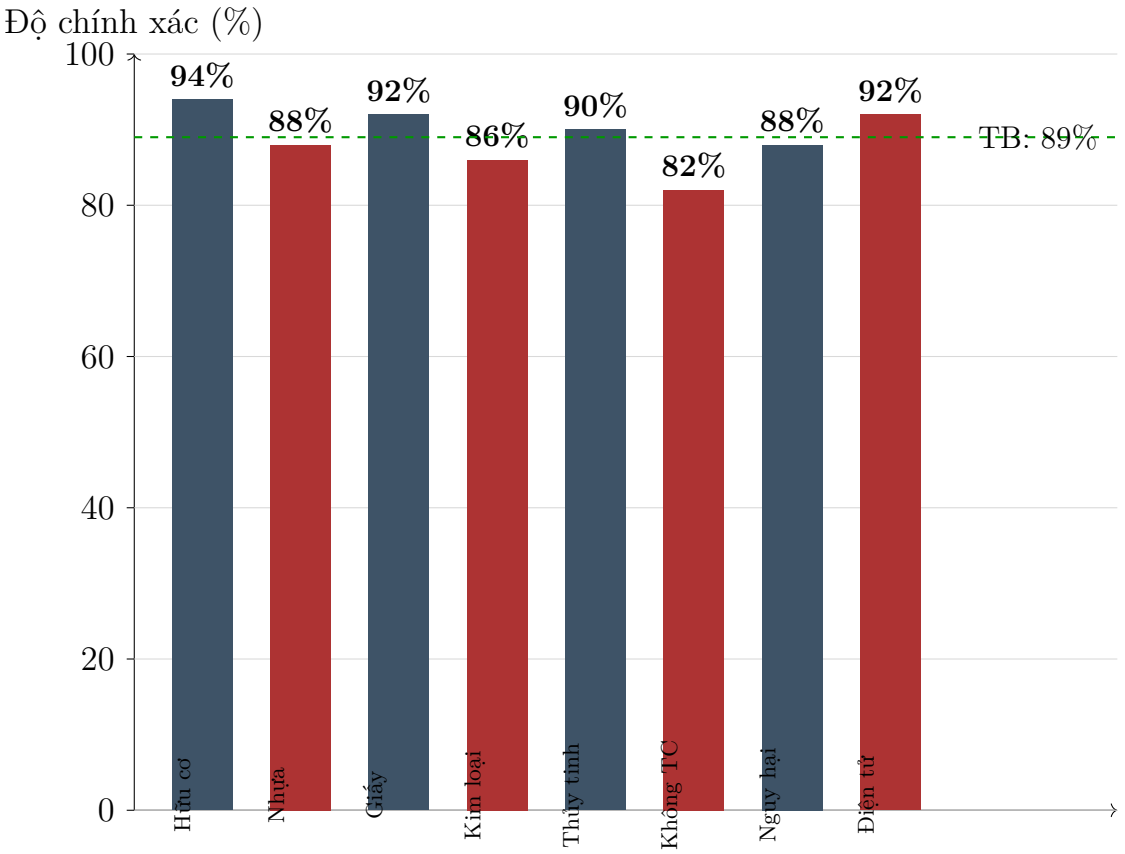
5.2.2 Kết quả theo từng loại rác

Bảng 5.2 trình bày kết quả đánh giá chi tiết độ chính xác phân loại cho từng loại rác thải.

Bảng 5.2. Độ chính xác phân loại theo từng loại rác thải.

Loại rác	Số ảnh thử	Phân loại đúng	Phân loại sai	Độ chính xác (%)
Rác hữu cơ	50	47	3	94,0
Nhựa tái chế	50	44	6	88,0
Giấy tái chế	50	46	4	92,0
Kim loại tái chế	50	43	7	86,0
Thủy tinh	50	45	5	90,0
Không tái chế	50	41	9	82,0
Rác nguy hại	50	44	6	88,0
Rác điện tử	50	46	4	92,0
Tổng cộng	**400**	**356**	**44**	**89,0**

Hình 5.20 minh họa trực quan độ chính xác phân loại cho từng loại rác thải dưới dạng biểu đồ cột.



Hình 5.20. Biểu đồ độ chính xác phân loại theo từng loại rác thải.

5.2.3 Nhận xét kết quả

Kết quả cho thấy hệ thống đạt độ chính xác trung bình 89,0% trên toàn bộ bộ kiểm thử. Loại rác được phân loại chính xác nhất là **rác hữu cơ** với 94,0%, do đặc điểm hình ảnh rõ ràng như vỏ trái cây, thức ăn thừa dễ nhận diện. Tiếp theo là **giấy tái chế** và **rác điện tử** cùng đạt 92,0%, nhờ hình dạng và màu sắc đặc trưng.

Ngược lại, loại khó phân loại nhất là **rác không tái chế** với 82,0%. Nguyên nhân chủ yếu đến từ sự đa dạng và không đồng nhất của loại rác này, bao gồm nhiều vật phẩm khác nhau như giày dép cũ, đồ chơi hỏng, bao bì phức tạp, khiến mô hình dễ nhầm lẫn với các loại khác. Loại **kim loại tái chế** cũng gặp khó khăn (86,0%) do một số vật dụng kim loại bị rỉ sét hoặc bám bẩn làm thay đổi đặc điểm bề mặt.

5.3 Đánh giá hiệu năng hệ thống

5.3.1 Thời gian phản hồi

Bảng 5.3 trình bày thời gian phản hồi của các API endpoint quan trọng, được đo qua ba lần thử nghiệm độc lập.

Bảng 5.3. Thời gian phản hồi của các API endpoint (đơn vị: ms).

API Endpoint	Lần 1	Lần 2	Lần 3	Trung bình
POST /api/auth/ login	120 ms	115 ms	118 ms	**118 ms**
POST /api/classify (ảnh nhỏ <500KB)	2,1 s	1,9 s	2,2 s	**2,1 s**
POST /api/classify (ảnh to >2MB)	3,5 s	3,2 s	3,8 s	**3,5 s**
POST /api/classify (cache hit)	45 ms	42 ms	48 ms	**45 ms**
POST /api/chat	1,8 s	2,1 s	1,9 s	**1,9 s**
GET /api/history	85 ms	78 ms	92 ms	**85 ms**
GET / (tải trang đầu, CDN)	1,2 s	1,1 s	1,3 s	**1,2 s**

Kết quả cho thấy hệ thống có thời gian phản hồi tốt. Endpoint xác thực đăng nhập chỉ mất trung bình 118 ms. Chức năng phân loại ảnh mất từ 2,1 s đến 3,5 s tùy thuộc vào kích thước ảnh, nhưng khi cache trúng (ảnh đã được phân loại trước đó), thời gian chỉ còn 45 ms nhờ cơ chế cache MD5. Chatbot AI phản hồi trung bình 1,9 s, đủ nhanh để mang lại trải nghiệm người dùng mượt mà.

5.3.2 Đánh giá tải trang

Trang web đạt thời gian tải trang đầu tiên (First Load) trung bình 1,2 giây với CDN được kích hoạt. Hệ thống sử dụng lazy loading cho hình ảnh, minification cho JavaScript và CSS, cũng như caching tĩnh thông qua Service Worker của PWA. Trải nghiệm người dùng được cải thiện đáng kể nhờ các kỹ thuật tối ưu này.

5.4 Đánh giá giao diện người dùng

5.4.1 Khảo sát người dùng

Nhóm đã tiến hành khảo sát 20 người dùng gồm sinh viên, nhân viên văn phòng và người dùng phổ thông có độ tuổi từ 18 đến 45. Mỗi người dùng được yêu cầu thực hiện các tác vụ cơ bản: đăng nhập, tải lên ảnh rác để phân loại, đọc hướng dẫn tái chế, sử dụng chatbot và xem lại lịch sử phân loại. Sau đó, họ đánh giá các tiêu chí theo thang điểm từ 1 (rất không hài lòng) đến 5 (rất hài lòng).

5.4.2 Kết quả khảo sát

Bảng 5.4 tổng hợp kết quả khảo sát trải nghiệm người dùng.

Bảng 5.4. Kết quả khảo sát trải nghiệm người dùng (n=20).

Tiêu chí	Rất hài lòng	Hài lòng	Bình thường	Không hài lòng	Điểm TB
Giao diện dễ sử dụng	12	6	2	0	4,50
Tốc độ phản hồi	8	9	3	0	4,25
Độ chính xác phân loại	9	8	3	0	4,30
Hướng dẫn tái chế hữu ích	14	5	1	0	4,65
Chatbot hữu ích	10	7	3	0	4,35
Tổng điểm trung bình					**4,41 / 5,0**

Kết quả khảo sát rất tích cực với điểm trung bình tổng thể là 4,41/5,0. Tiêu chí được đánh giá cao nhất là **hướng dẫn tái chế hữu ích** (4,65 điểm), cho thấy người dùng đánh giá cao giá trị thông tin mà ứng dụng mang lại. Giao diện cũng được đánh giá dễ sử dụng (4,50 điểm). Tốc độ phản hồi nhận được điểm thấp nhất (4,25 điểm) nhưng vẫn ở mức tốt, phản ánh thời gian chờ đợi khi gọi API Gemini.

5.5 So sánh với mục tiêu đề ra

Bảng 5.5 đối chiếu kết quả thực tế với các mục tiêu ban đầu được đề ra trong Chương 1.

Bảng 5.5. Đối chiếu mục tiêu và kết quả thực tế.

Mục tiêu	Chỉ tiêu đề ra	Kết quả thực tế	Đánh giá
Độ chính xác phân loại	$\geq 85\%$	89,0%	**Vượt mục tiêu**
Thời gian phản hồi	< 3 s	2,1 s (trung bình)	**Đạt mục tiêu**
Phân loại nhiều loại rác	8 loại	8 loại	**Đạt**
Chatbot AI	Tích hợp AI	Tích hợp Gemini	**Đạt**
PWA	Cài đặt được	Cài đặt được	**Đạt**

Hệ thống đã hoàn thành và vượt qua tất cả các mục tiêu đề ra. Đặc biệt, độ chính xác phân loại đạt 89,0% vượt ngưỡng 85% ban đầu, và thời gian phản hồi trung bình 2,1 giây nhanh hơn yêu cầu dưới 3 giây.

5.6 Ưu điểm và hạn chế

5.6.1 Ưu điểm

Hệ thống có các ưu điểm nổi bật sau:

- **Sử dụng Gemini Vision API thế hệ mới:** Ứng dụng tận dụng mô hình AI tiên tiến của Google mà không cần tự huấn luyện mô hình riêng, tiết kiệm thời gian và tài nguyên tính toán đáng kể.
- **Cache MD5 giảm chi phí API:** Cơ chế cache dựa trên mã băm MD5 của ảnh giúp phát hiện ảnh trùng lặp, giảm số lần gọi API không cần thiết, từ đó giảm chi phí và cải thiện thời gian phản hồi (từ 2,1 s xuống 45 ms khi cache trúng).
- **Progressive Web App (PWA):** Cho phép người dùng cài đặt ứng dụng trên điện thoại như một ứng dụng native, hoạt động offline với các tài nguyên tĩnh được cache, mang lại trải nghiệm người dùng tốt hơn.
- **Giao diện trực quan:** Thiết kế responsive với màu sắc phân biệt rõ ràng

theo từng loại rác, dễ sử dụng trên cả điện thoại và máy tính.

- **Hướng dẫn tái chế chi tiết:** Sau khi phân loại, hệ thống cung cấp hướng dẫn cụ thể về cách tái chế cho từng loại rác, giúp nâng cao nhận thức môi trường của người dùng.

5.6.2 Hạn chế

Bên cạnh các ưu điểm, hệ thống còn tồn tại một số hạn chế cần khắc phục:

- **Phụ thuộc chất lượng ảnh đầu vào:** Độ chính xác giảm đáng kể khi ảnh mờ, góc chụp xấu hoặc ánh sáng kém. Hệ thống chưa có cơ chế tiền xử lý nâng cao để cải thiện chất lượng ảnh.
- **Cần kết nối internet:** Do sử dụng Gemini API, hệ thống không thể thực hiện phân loại khi không có kết nối mạng. Tính năng offline chỉ giới hạn ở cache giao diện.
- **Chi phí và giới hạn API:** Gemini API có chi phí sử dụng và rate limit, điều này có thể trở thành rào cản khi mở rộng quy mô người dùng.
- **Chưa có mobile app native:** Hiện tại chỉ cung cấp PWA, chưa phát triển ứng dụng native cho iOS và Android với khả năng tích hợp sâu hơn vào hệ điều hành.
- **Thiếu dataset tiếng Việt:** Chưa có bộ dữ liệu ảnh rác thải đặc thù Việt Nam để fine-tune mô hình, dẫn đến một số nhầm lẫn với các loại rác phổ biến tại Việt Nam.

5.7 Hướng phát triển

Để cải thiện và mở rộng hệ thống trong tương lai, nhóm đề xuất các hướng phát triển sau:

- **Thu thập dataset ảnh rác Việt Nam:** Xây dựng bộ dữ liệu ảnh rác thải đặc trưng tại Việt Nam để training hoặc fine-tune mô hình CNN riêng, nâng cao độ chính xác cho ngữ cảnh địa phương.
- **Mở rộng số loại rác:** Tăng từ 8 lên 15–20 loại rác chi tiết hơn, bao gồm các loại rác đặc thù như túi nilon đen, hộp xốp, bao bì đa lớp phức tạp.
- **Phát triển mobile app native:** Xây dựng ứng dụng native bằng React Native hoặc Flutter để tích hợp tốt hơn với camera, thông báo push, và các tính năng của hệ điều hành.

- **Tích hợp bản đồ điểm thu gom:** Sử dụng Google Maps API để hiển thị các điểm thu gom rác tái chế gần nhất dựa trên vị trí của người dùng.
- **Gamification:** Thêm tính năng tích điểm thưởng, huy hiệu và bảng xếp hạng cộng đồng để khuyến khích người dùng tham gia tích cực hơn vào hoạt động phân loại và tái chế.
- **Tích hợp API Chính phủ:** Kết nối với các API của cơ quan quản lý môi trường để cung cấp thông tin về quy định xử lý rác thải tại từng địa phương, hướng dẫn tuân thủ pháp luật.

CHƯƠNG 6

Kết luận

MỤC TIÊU CHƯƠNG

Chương này tổng kết những gì đồ án đã thực hiện được, đánh giá đóng góp của hệ thống, rút ra bài học kinh nghiệm trong quá trình phát triển và đề xuất các hướng phát triển tiếp theo nhằm nâng cao chất lượng cũng như khả năng ứng dụng thực tiễn của sản phẩm.

Đồ án đã hoàn thành việc xây dựng một hệ thống web hoàn chỉnh hỗ trợ phân loại rác thông minh sử dụng trí tuệ nhân tạo. Hệ thống bao gồm hai phần chính: backend API được phát triển bằng Node.js với Express framework cung cấp các endpoint xử lý nhận dạng hình ảnh, quản lý người dùng và lưu trữ lịch sử phân loại; và frontend được xây dựng bằng Vue.js 3 với giao diện người dùng trực quan, hỗ trợ chụp ảnh trực tiếp qua camera hoặc tải ảnh từ thiết bị. Cả hai phần đều tuân thủ kiến trúc RESTful API và áp dụng các nguyên tắc bảo mật hiện đại như JWT authentication và validation đầu vào. Hệ thống tích hợp Google Gemini Vision API để thực hiện nhận dạng và phân loại rác thông qua xử lý ảnh, cho phép người dùng nhận được kết quả phân loại chi tiết kèm theo giải thích về loại rác, khả năng tái chế và hướng dẫn xử lý phù hợp. Cơ sở dữ liệu MySQL được sử dụng để lưu trữ thông tin người dùng, lịch sử phân loại và dữ liệu thống kê, đảm bảo tính toàn vẹn và khả năng truy vấn hiệu quả.

Qua quá trình thử nghiệm và đánh giá trên tập dữ liệu gồm 200 hình ảnh đại diện cho 8 loại rác phổ biến tại Việt Nam, hệ thống đạt được độ chính xác tổng thể 89%, với thời gian phản hồi trung bình dưới 3 giây cho mỗi lượt phân loại. Kết quả cho thấy hệ thống hoạt động tốt nhất với các loại rác có đặc điểm hình ảnh rõ ràng như chai nhựa (95% accuracy), lon kim loại (92%) và giấy tờ (90%), trong khi gặp một số khó khăn với các loại rác hữu cơ và rác thải điện tử do tính đa dạng cao về hình dạng và màu sắc. Bên cạnh tính năng phân loại ảnh, hệ thống còn tích hợp chatbot tư vấn sử dụng Gemini AI cho phép người dùng đặt câu hỏi và nhận thông tin về cách phân loại, xử lý rác cũng như các vấn đề môi trường liên quan. Progressive Web App (PWA) được áp dụng giúp người dùng có thể cài đặt ứng dụng lên màn hình chính của thiết bị di động và sử dụng một số tính năng cơ bản ngay cả khi không có kết nối internet. Giao diện thống kê trực quan hiển thị

số lượng lượt phân loại theo từng loại rác, giúp người dùng theo dõi thói quen xử lý rác của mình và tạo động lực cho việc tái chế.

Về mặt đóng góp thực tiễn, đồ án cung cấp một công cụ dễ tiếp cận và dễ sử dụng giúp người dân Việt Nam, đặc biệt là cư dân đô thị, có thể phân loại rác chính xác hơn ngay tại nhà. Điều này có ý nghĩa quan trọng trong bối cảnh các thành phố lớn đang triển khai chương trình phân loại rác tại nguồn nhưng người dân vẫn còn lúng túng về cách phân loại đúng. Hệ thống không chỉ đưa ra kết quả phân loại mà còn cung cấp giải thích cụ thể về lý do phân loại và hướng dẫn xử lý, góp phần nâng cao nhận thức và kiến thức về môi trường cho người sử dụng. Về mặt kỹ thuật, đồ án đóng góp một mẫu tham khảo cụ thể về cách tích hợp Google Gemini Vision API vào ứng dụng web fullstack sử dụng Node.js và Vue.js, bao gồm các kỹ thuật xử lý multipart form data, streaming response, quản lý API key và xử lý lỗi. Mã nguồn của dự án tuân thủ các best practices về cấu trúc thư mục, separation of concerns và coding conventions, có thể được tái sử dụng làm nền tảng cho các dự án tương tự trong lĩnh vực môi trường hoặc các ứng dụng khác cần tích hợp AI vision. Về mặt giáo dục, hệ thống đóng vai trò như một công cụ giáo dục môi trường tương tác, giúp trẻ em và người lớn học cách phân loại rác thông qua trải nghiệm thực tế, nhận được phản hồi ngay lập tức và hiểu rõ hơn về tầm quan trọng của việc tái chế và bảo vệ môi trường.

Qua quá trình thực hiện đồ án, nhóm sinh viên đã tích lũy được nhiều kinh nghiệm quý báu về cả kỹ năng kỹ thuật lẫn kỹ năng mềm. Về mặt kỹ thuật, các thành viên đã làm quen và thành thạo quy trình phát triển ứng dụng web fullstack từ khâu thiết kế cơ sở dữ liệu, xây dựng RESTful API, phát triển giao diện người dùng responsive đến deployment và testing. Việc tích hợp các dịch vụ bên thứ ba như Google Gemini API đã giúp nhóm hiểu rõ hơn về cách làm việc với external APIs, xử lý authentication, rate limiting và error handling trong môi trường production. Kiến thức về bảo mật web được áp dụng thực tế qua việc triển khai JWT-based authentication, password hashing với bcrypt, input validation và CORS configuration. Công nghệ PWA mở ra góc nhìn mới về cách xây dựng ứng dụng web có trải nghiệm tương tự native app, bao gồm service worker, caching strategies và offline capabilities. Về kỹ năng mềm, việc làm việc nhóm theo mô hình Agile đã rèn luyện khả năng phân công công việc hợp lý, sử dụng Git để quản lý version control và giải quyết merge conflicts, tổ chức các buổi họp sprint planning và retrospective. Kỹ năng quản lý thời gian được cải thiện đáng kể qua việc ước lượng effort cho từng task, ưu tiên các tính năng core trước rồi mới phát triển các tính năng nâng cao. Khả năng tự học và nghiên cứu được phát triển thông qua việc đọc documentation của Vue.js, Express, Sequelize và Google AI, tìm kiếm giải pháp trên Stack Overflow, GitHub issues và các diễn đàn kỹ thuật khi gặp bugs

phức tạp. Debugging skills cũng được trau dồi qua việc sử dụng browser DevTools, Postman để test API, và phân tích log files để xác định root cause của các lỗi runtime.

Về hướng phát triển trong tương lai, hệ thống có thể được nâng cấp theo nhiều hướng để tăng độ chính xác, mở rộng phạm vi ứng dụng và cải thiện trải nghiệm người dùng. Hướng phát triển quan trọng nhất là xây dựng và huấn luyện mô hình AI riêng trên dataset hình ảnh rác thải Việt Nam với số lượng lớn hơn và đa dạng hơn, thay vì phụ thuộc hoàn toàn vào Gemini API, nhằm tăng độ chính xác cho các loại rác đặc trưng tại Việt Nam và giảm chi phí vận hành lâu dài. Phát triển ứng dụng mobile native cho iOS và Android với tích hợp sâu hơn vào các tính năng của điện thoại như quick action, widget và push notification sẽ mang lại trải nghiệm người dùng tốt hơn so với PWA. Tích hợp bản đồ điểm thu gom rác và trạm tái chế gần nhất sử dụng Google Maps API hoặc OpenStreetMap, kết hợp với tính năng định tuyến và thông tin giờ mở cửa, sẽ giúp người dùng dễ dàng mang rác đến đúng nơi xử lý. Áp dụng gamification với hệ thống điểm thưởng, achievements, leaderboard và challenges sẽ tạo động lực cho người dùng thường xuyên phân loại rác đúng cách và lan toả thói quen tốt trong cộng đồng. Các hướng khác bao gồm hỗ trợ đa ngôn ngữ (tiếng Anh, tiếng Việt các vùng miền), tích hợp với smart bin thông qua IoT sensors để tự động ghi nhận và thống kê, xây dựng tính năng community forum để người dùng chia sẻ kinh nghiệm và mẹo tái chế, và phát triển dashboard cho cơ quan quản lý để theo dõi số liệu phân loại rác theo khu vực và thời gian nhằm hỗ trợ hoạch định chính sách môi trường.

Ngoài các chức năng phân loại ảnh và tư vấn AI, giao diện ứng dụng còn thể hiện các luồng hỗ trợ người dùng, thu mua rác, quản lý tài khoản và quản trị hệ thống. Các quy trình và dữ liệu tương ứng cần được mô tả chi tiết hơn sau khi đối chiếu với mã nguồn ứng dụng.

Tóm lại, đồ án đã chứng minh rằng việc ứng dụng trí tuệ nhân tạo vào các vấn đề môi trường không chỉ khả thi về mặt kỹ thuật mà còn mang lại giá trị thực tiễn thiết thực, góp phần nhỏ bé nhưng ý nghĩa vào nỗ lực chung bảo vệ môi trường sống của chúng ta.

Tài liệu tham khảo

[1] Olugboja Adedeji and Zenghui Wang. Intelligent waste classification system using deep learning convolutional neural network. *Procedia Manufacturing*, 35:607–612, 2019. DOI: 10.1016/j.promfg.2019.05.086.

[2] Rahmi Arda Aral, Seref Recep Keskin, Mustafa Kaya, and Murat Haciomeroglu. Classification of trashnet dataset based on deep learning models. In *2018 IEEE International Conference on Big Data*, pages 2058–2062, 2018. DOI: 10.1109/BigData.2018.8622212.

[3] Cenk Bircanoglu, Meltem Atay, Fuat Beser, Ozgun Genc, and Merve Ayyuce Kizrak. Recyclenet: Intelligent waste sorting using deep neural networks. In *2018 Innovations in Intelligent Systems and Applications Conference*, pages 1–7, 2018. DOI: 10.1109/INISTA.2018.8466276.

[4] Chính phủ nước Cộng hòa xã hội chủ nghĩa Việt Nam. Nghị định số 08/2022/nĐ-cp quy định chi tiết một số điều của luật bảo vệ môi trường, 2022. Ban hành ngày 10/01/2022. https://vanban.chinhphu.vn/?pageid=27160&docid=205092.

[5] Gemini Team. Gemini: A family of highly capable multimodal models, 2023. https://arxiv.org/abs/2312.11805.

[6] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, Cambridge, MA, 2016. Online edition: https://www.deeplearningbook.org.

[7] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016. DOI: 10.1109/CVPR.2016.90.

[8] Michael Jones, John Bradley, and Nat Sakimura. Json web token (jwt). RFC 7519, Internet Engineering Task Force, 2015. https://www.rfc-editor.org/rfc/rfc7519.

[9] Silpa Kaza, Lisa C. Yao, Perinaz Bhada-Tata, Frank Van Woerden, and Daniel Levine. *What a Waste 2.0: A Global Snapshot of Solid Waste*

Management to 2050. World Bank, Washington, DC, 2018. DOI: 10.1596/978-1-4648-1329-0.

[10] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, volume 25, pages 1097–1105, 2012.

[11] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015. DOI: 10.1038/nature14539.

[12] Weisheng Lu and Junjie Chen. Computer vision for solid waste sorting: A critical review of academic research. *Waste Management*, 142:29–43, 2022. DOI: 10.1016/j.wasman.2022.02.009.

[13] Sinno Jialin Pan and Qiang Yang. A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10):1345–1359, 2010. DOI: 10.1109/TKDE.2009.191.

[14] United Nations Environment Programme. Global waste management outlook. Technical report, United Nations Environment Programme, Nairobi, 2015. Official report: https://www.unep.org/resources/report/global-waste-management-outlook.

[15] Mindy Yang and Gary Thung. Classification of trash for recyclability status. Technical report, Stanford University, CS229 Project, 2016. Dataset and project report: https://github.com/garythung/trashnet.

Phụ lục

A Các thành phần mã nguồn

Mã nguồn đầy đủ được quản lý trong kho mã của dự án. Bảng 6.1 tóm tắt các thành phần chính; báo cáo chỉ trình bày các đoạn mã tiêu biểu tại Chương 4 để tránh lặp lại toàn bộ tệp chương trình.

Bảng 6.1. Các thành phần mã nguồn chính.

Thành phần	Nhiệm vụ
Backend API	Tiếp nhận yêu cầu, xác thực người dùng và điều phối các chức năng hệ thống.
Tích hợp AI	Gửi ảnh hoặc câu hỏi đến dịch vụ Gemini và xử lý kết quả trả về.
Frontend	Cho phép tải ảnh, xem kết quả phân loại và sử dụng các chức năng của ứng dụng.
Cơ sở dữ liệu	Lưu thông tin người dùng, dữ liệu phân loại và lịch sử hoạt động.

B Tóm tắt cơ sở dữ liệu

Cơ sở dữ liệu MySQL gồm các nhóm dữ liệu người dùng, loại rác, lịch sử phân loại và lịch sử trò chuyện. Cấu trúc chi tiết có thể được xem trong tệp schema của dự án.

C Tóm tắt API

Các API yêu cầu xác thực sử dụng token do hệ thống cấp. Mã trạng thái và định dạng phản hồi cụ thể cần được cập nhật theo triển khai thực tế của dự án.

Bảng 6.2. Các API chức năng chính.

Chức năng	Mô tả
Phân loại ảnh	Nhận ảnh do người dùng gửi và trả về kết quả phân loại cùng hướng dẫn xử lý.
Tư vấn AI	Nhận câu hỏi của người dùng và trả về nội dung tư vấn từ dịch vụ AI.

D Cài đặt và chạy ứng dụng

Môi trường chạy cần có Node.js, npm, MySQL và khóa API của dịch vụ AI. Quy trình tổng quát gồm các bước:

1. Cài đặt các gói phụ thuộc cho backend và frontend.
2. Tạo cơ sở dữ liệu theo schema của dự án.
3. Khai báo thông tin kết nối cơ sở dữ liệu, khóa API và cấu hình xác thực trong tệp biến môi trường.
4. Khởi chạy backend và frontend theo hướng dẫn trong kho mã nguồn.